

AgentDropout: Dynamic Agent Elimination for Token-Efficient and High-Performance LLM-Based Multi-Agent Collaboration

Zhexuan Wang¹ Yutong Wang¹ Xuebo Liu^{1*} Liang Ding²

Miao Zhang³ Jie Liu⁴ Min Zhang¹

¹Institute of Computing and Intelligence, Harbin Institute of Technology, Shenzhen, China

²The University of Sydney, Sydney, Australia

³School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen, China

⁴State Key Lab of Smart Farm Technologies and Systems, Harbin Institute of Technology, Harbin, China

{wangzhexuan,wangyutong}@stu.hit.edu.cn, liangding.liam@gmail.com

{liuxuebo,zhangmiao,jieliu,zhangmin2021}@hit.edu.cn

Abstract

Multi-agent systems (MAS) based on large language models (LLMs) have demonstrated significant potential in collaborative problem-solving. However, they still face substantial challenges of low communication efficiency and suboptimal task performance, making the careful design of the agents' communication topologies particularly important. Inspired by the management theory that roles in an efficient team are often dynamically adjusted, we propose **AgentDropout**, which identifies redundant agents and communication across different communication rounds by optimizing the adjacency matrices of the communication graphs and eliminates them to enhance both token efficiency and task performance. Compared to state-of-the-art methods, AgentDropout achieves an average reduction of 21.6% in prompt token consumption and 18.4% in completion token consumption, along with a performance improvement of 1.14 on the tasks. Furthermore, the extended experiments demonstrate that AgentDropout achieves notable domain transferability and structure robustness, revealing its reliability and effectiveness. We release our code at <https://github.com/wangzx1219/AgentDropout>.

1 Introduction

In recent years, LLM-based (Achiam et al., 2023; Team et al., 2024) agent technologies have achieved robust performance across a wide range of tasks. Prior works have discovered that LLM-based agents (Liang et al., 2024; Wang et al., 2025a; Du et al., 2024; Wang et al., 2024b; Zhuge et al., 2024; Wang et al., 2024a) can emulate human-like collaboration through mutual communication, leading to enhanced performance in tasks and forming MAS that resemble cooperative human teams.

However, MAS often suffers from high token consumption and inefficiency (Li et al., 2024). This

* Corresponding Author

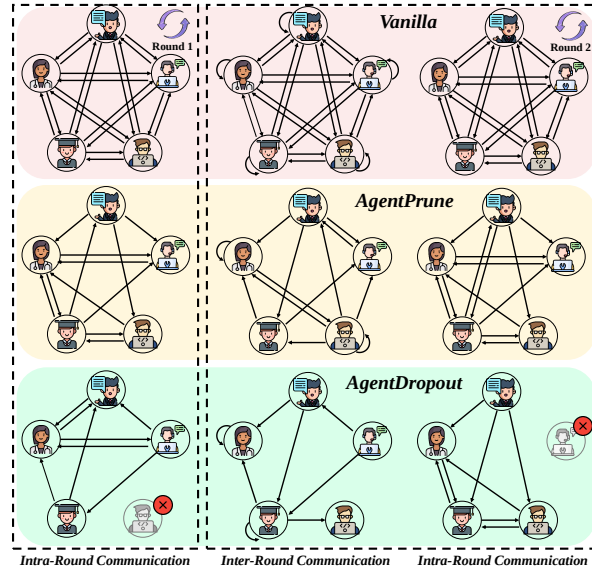


Figure 1: An overview of AgentDropout, in comparison with vanilla MAS and AgentPrune (Zhang et al., 2025), highlighting its dynamic adjustments of participant roles in each discussion to enhance cooperation effectiveness and efficiency.

is primarily due to the frequent generation and communication among multiple agents, which incurs substantial token overhead. Existing methods mainly apply simple agent profile evolution (Chen et al., 2024a) or memory evolution (Wang et al., 2025b; Qian et al., 2024a,b; Gao et al., 2024), which fails to address the core problem of multi-agent interaction optimization. To address this issue, Zhuge et al. (2024) proposed the pruning of inter-graph communication edges, while Zhang et al. (2025) defined communication redundancy and introduced a pruning method AgentPrune within MAS. Specifically, existing methods employ trainable communication graph masks to iteratively identify critical message exchanges and prune redundant edges, thereby generating a sparser and more efficient communication graph. However, constrained by the unchanged roles participating in the discussion in each round in MAS, the existing method applies a uniform pruning strategy across

AgentDropout: 面向Token高效与高性能基于大语言模型的多智能体协作的动态智能体剔除

王哲轩¹ 王雨桐¹ 刘学博^{1*} 丁亮² 张苗³ 刘杰⁴ 张敏¹

¹哈尔滨工业大学（深圳）计算与智能学院²澳大利亚悉尼大学³哈尔滨工业大学（深圳）计算机科学与技术学院⁴哈尔滨工业大学智能农业技术与系统国家实验室

{wangzhexuan,wangyutong}@stu.hit.edu.cn, liangding.liam@gmail.com

{liuxuebo,zhangmiao,jieliu,zhangmin2021}@hit.edu.cn

摘要

基于大语言模型（LLMs）的多智能体系统（MAS）在协作问题解决中展现了巨大潜力。然而，它们仍面临通信效率低和任务性能不佳的严峻挑战，这使得仔细设计智能体的通信拓扑尤为重要。受“高效团队中的角色通常会动态调整”这一管理理论的启发，我们提出了 **AgentDropout**。该方法通过优化通信图的邻接矩阵，识别并消除冗余智能体以及不同通信轮次中的冗余通信，从而同时提升令牌效率和任务性能。与最先进的方法相比，AgentDropout 平均降低了 21.6% 的提示令牌消耗和 18.4% 的补全令牌消耗，并在任务上实现了 1.14 的性能提升。此外，扩展实验表明，AgentDropout 具有显著的领域可迁移性和结构鲁棒性，揭示了其可靠性和有效性。我们的代码已发布于

github.com/wangzx1219/AgentDropout.

1 引言

近年来，基于大语言模型的（Achiam et al., 2023; Team et al., 2024）智能体技术在广泛的任务中实现了稳健的性能。先前的研究发现，基于大语言模型的智能体（Liang et al., 2024; Wang et al., 2025a; Du et al., 2024; Wang et al., 2024b; Zhuge et al., 2024; Wang et al., 2024a）能够通过相互沟通模拟人类的协作，从而在任务中提升性能，并形成了类似于合作型人类团队的多智能体系统。

然而，多智能体系统往往存在高令牌消耗和低效率的问题（Li et al., 2024）。这一点

* 通讯作者

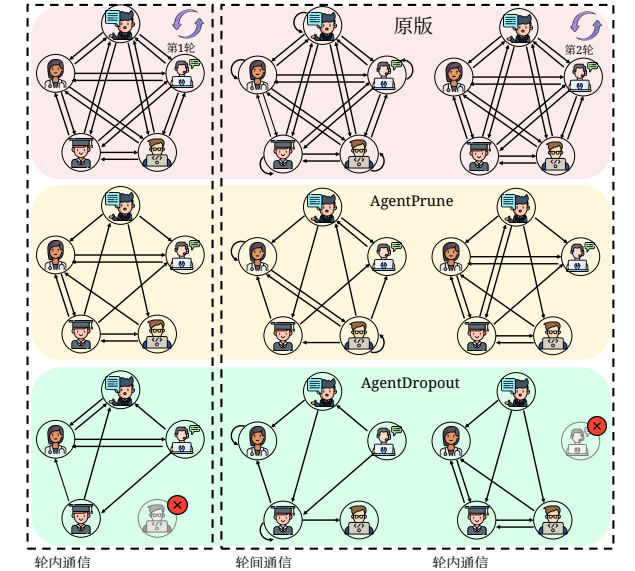


图 1: AgentDropout 的概览，与原始多智能体系统（vanilla MAS）和 AgentPrune (Zhang et al., 2025) 进行对比，突出其在每次讨论中对参与者角色进行动态调整，以增强协作有效性和效率。

这主要是由于多个智能体之间的频繁生成和通信，导致产生大量令牌开销。现有方法主要应用简单的智能体档案演化（Chen et al., 2024a）或记忆演化（Wang et al., 2025b; Qian et al., 2024a,b; Gao et al., 2024），这未能解决多智能体交互优化这一核心问题。为了解决该问题，Zhuge 等人 (2024) 提出了图间通信边的剪枝方法，而 Zhang 等 (2025) 定义了通信冗余，并在 MAS 中引入了一种剪枝方法 AgentPrune。具体而言，现有方法采用可训练通信图掩码，迭代地识别关键消息交换并剪除冗余边，从而生成更稀疏、更高效的通信图。然而，受限于 MAS 中每轮次参与讨论的固定角色，现有方法在所有通信轮次中应用统一的剪枝策略，

multiple communication rounds, which may limit its effectiveness in improving system efficiency and performance.

Drawing an analogy between MAS and human teams, it is evident that dynamic role allocation and team collaboration are pivotal factors in enhancing decision-making efficiency and overall performance in high-performing teams. Locke (2011) have pointed out that an effective team must adaptively adjust the roles and responsibilities of its members based on task requirements, Kozlowski and Ilgen (2006) further emphasizing the critical role of such dynamic adjustments in improving team decision-making efficacy. Inspired by these insights, we propose AgentDropout, as shown in figure 1. Specifically, AgentDropout iteratively updates the communication topology using a small amount of data, simulating the behavior of dynamic role assignment during multi-round discussions. This process consists of two steps: (1) **Node Dropout**: Similar to how team members assume different responsibilities at various stages, we train the edge weights and compute the degree of nodes in this step. By selectively retaining or removing certain nodes across different rounds, the flexibility and efficiency of the entire system are enhanced. (2) **Edge Dropout**: We optimize the communication and collaboration mechanisms within teams by training edge weights and pruning redundant or low-contribution edges to refine the connections between nodes. This approach not only reduces the computational complexity but also enhances the effectiveness of information propagation.

Experiments conducted on a variety of tasks, including reasoning, mathematics, and code generation, demonstrate that AgentDropout outperforms different kinds of MAS and AgentPrune in terms of performance and token efficiency. Our main contributions are as follows:

- We observe that in MAS with both intra- and inter-round characteristics, involving different roles in distinct rounds of discussion is more efficient and leads to better performance compared to engaging the same roles in every communication round.
- We propose AgentDropout, a simple and flexible optimization method for MAS. It adjusts the overall communication topology through two steps: **Node Dropout** and **Edge Dropout**.
- We observe that for larger-scale and more

powerful LLMs, the performance of MAS designed on top of them can still be improved through interaction. AgentDropout can further enhance efficiency, domain transferability, and structural robustness.

2 Background

2.1 LLM-based MAS as Graph

Recently, Chan et al. (2024); Jiang et al. (2023); Wu et al. (2023) have proposed representing MAS using directed communication graphs, denoted as $G = (V, E, F)$. $V = \{v_i\}_{i=1}^N$ is a set of nodes, and each node represents an agent. $E \subset V \times V$ is a set of edges, where each edge $(v_i, v_j) \in E$ ($i \neq j$) denotes the communication path from agent v_i to agent v_j . $F = \{f_i\}_{i=1}^N$ is a set of mapping functions that abstract the agent inference process. Each node v_i receives a query q and the outputs of its in-neighbors $Z_i = \{o_j \mid (v_j, v_i) \in E\}$ to produce its own output $o_i = f_i(q, Z_i)$.

For some complex tasks, the entire communication graph needs to be run over multiple rounds to obtain the final answer, with each round operating on the communication graph defined as: $G^{(t)} = (V^{(t)}, E_{\text{intra}}^{(t)}, F^{(t)})$, where $E_{\text{intra}}^{(t)}$ denotes the **intra-round edges** which only connects nodes in the same round. Besides, the output of any node in step $t - 1$ can be passed to any node in step t , forming **inter-round edges** $E_{\text{inter}}^{(t)} \subset V^{(t-1)} \times V^{(t)}$ between graph $G^{(t-1)}$ and $G^{(t)}$. The subgraphs for all steps $\{G^{(1)}, G^{(2)}, \dots, G^{(T)}\}$ are combined using $E_{\text{inter}}^{(t)}$ to form a multi-round communication graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{F})$, where $\mathcal{V} = \bigcup_t V^{(t)}$, $\mathcal{E} = \left(\bigcup_t E_{\text{intra}}^{(t)}\right) \cup \left(\bigcup_t E_{\text{inter}}^{(t)}\right)$, and $\mathcal{F} = \bigcup_t F^{(t)}$.

2.2 Communication Redundancy in MAS

MAS often incurs high token consumption associated with the frequent communications between agents. Zhang et al. (2025) defined communication redundancy in MAS – excessive information flow that does not help improve performance. Considering a subgraph $\mathcal{G}_{\text{sub}} = (\mathcal{V}, \mathcal{E}', \mathcal{F})$, where $\mathcal{E}' \subseteq \mathcal{E}$, communication redundancy is considered to exist if $\mu(\mathcal{G}_{\text{sub}}) \geq \mu(\mathcal{G})$, where $\mu(\cdot)$ is the task-specified utility function evaluating the performance of the MAS. Redundant edges are those in $\mathcal{E} \setminus \mathcal{E}'$.

AgentPrune is an approach to reduce communication redundancy. By iteratively optimizing two trainable matrix masks (intra- and inter-round), AgentPrune parameterizes and prunes the edges in

这可能限制其在提升系统效率和性能方面的有效性。

将多智能体系统（MAS）与人类团队进行类比，可以明显看出，动态角色分配和团队协作是提升高效团队中决策效率和整体表现的关键因素。洛克（2011）指出，一个高效的团队必须根据任务要求自适应地调整其成员的角色和职责，而科兹洛夫斯基和伊尔根（2006年）进一步强调了这种动态调整在提升团队决策效能中的关键作用。受这些见解启发，我们提出了AgentDropout，如图1所示。具体来说，AgentDropout使用少量数据迭代更新通信拓扑，模拟多轮讨论期间动态角色分配的行为。该过程包含两个步骤：（1）节点丢弃：类似团队成员在不同阶段承担不同职责的做法，我们在此步骤中训练边权重并计算节点度。通过在每轮次中选择性保留或删除某些节点，整个系统的灵活性和效率得到增强。（2）边丢弃：我们通过训练边权重并剪除冗余或低贡献的边，来优化团队内的通信和协作机制，从而精炼节点间的连接。这种方法不仅降低了计算复杂度，还增强了信息传播的有效性。

在多种任务（包括推理、数学和代码生成）上进行的实验表明，与不同类型的多智能体系统（MAS）和AgentPrune相比，AgentDropout在性能和令牌效率方面均表现更优。我们的主要贡献如下：

- 我们观察到，在同时具备轮内和轮间特征的多智能体系统中，在不同轮次的讨论中引入不同角色，比在每个通信轮次中都让相同角色参与更为高效，并能带来更好的性能。
- 我们提出了AgentDropout，一种用于多智能体系统的简单且灵活的优化方法。它通过两步调整整体通信拓扑：节点丢弃和边丢弃。
- 我们观察到，对于更大规模且更

凭借功能强大的大语言模型，基于其构建的多智能体系统的性能仍可通过交互得到提升。AgentDropout可进一步提升效率、领域可迁移性以及结构鲁棒性。

2 背景

2.1 将基于大语言模型的多智能体系统建模为图

近年来，Chan等人（2024年）、Jiang等人（2023年）以及Wu等人（2023年）提出了使用有向通信图来表示多智能体系统，记为 $G = (V, E, F)$ 。 $V = \{v_i\}_{i=1}^N$ 是节点的集合，每个节点代表一个智能体。 $E \subset V \times V$ 是边的集合，其中每条边 $(v_i, v_j) \in E$ ($i \neq j$)表示从智能体 v_i 到智能体 v_j 的通信路径。 $F = \{f_i\}_{i=1}^N$ 是映射函数的集合，用于抽象智能体推理过程。每个节点 v_i 接收一个查询 q 及其入邻居 $Z_i = \{o_j \mid (v_j, v_i) \in E\}$ 的输出，以产生其自身的输出 $o_i = f_i(q, Z_i)$ 。

对于某些复杂任务，整个通信图需要运行多个轮次以获得最终答案，其中每个轮次操作于定义为 $G^{(t)} = (V^{(t)}, E_{\text{intra}}^{(t)}, F^{(t)})$ 的通信图上，其中 $E_{\text{intra}}^{(t)}$ 表示仅在相同轮次内连接节点的轮内边。此外，步骤 $t - 1$ 中任何节点的输出都可以传递给步骤 t 中的任何节点，从而在图 $G^{(t-1)}$ 和 $G^{(t)}$ 之间形成轮间边 $E_{\text{inter}}^{(t)} \subset V^{(t-1)} \times V^{(t)}$ 。所有步骤 $\{G^{(1)}, G^{(2)}, \dots, G^{(T)}\}$ 的子图使用 $E_{\text{inter}}^{(t)}$ 组合，形成一个多轮次通信图 $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{F})$ ，其中 $\mathcal{V} = \bigcup_t V^{(t)}$ ， $\mathcal{E} = \left(\bigcup_t E_{\text{intra}}^{(t)}\right) \cup \left(\bigcup_t E_{\text{inter}}^{(t)}\right)$ ，并且 $\mathcal{F} = \bigcup_t F^{(t)}$ 。

2.2 多智能体系统中的通信冗余

多智能体系统（MAS）通常会因智能体之间的频繁通信而产生高昂的令牌消耗。Zhang等人（2025年）定义了多智能体系统中的通信冗余——即那些不会帮助提升性能的信息流冗余。考虑一个子图 $\mathcal{G}_{\text{sub}} = (\mathcal{V}, \mathcal{E}', \mathcal{F})$ ，其中 $\mathcal{E}' \subseteq \mathcal{E}$ ，如果 $\mu(\mathcal{G}_{\text{sub}}) \geq \mu(\mathcal{G})$ ，则认为存在通信冗余，其中 $\mu(\cdot)$ 是用于评估多智能体系统性能的由任务指定的效用函数。冗余边是指 $\mathcal{E} \setminus \mathcal{E}'$ 中的边。

AgentPrune是一种减少通信冗余的方法。通过迭代优化两个可训练的矩阵掩码（轮内和轮间），AgentPrune参数化并剪枝通信图中的边

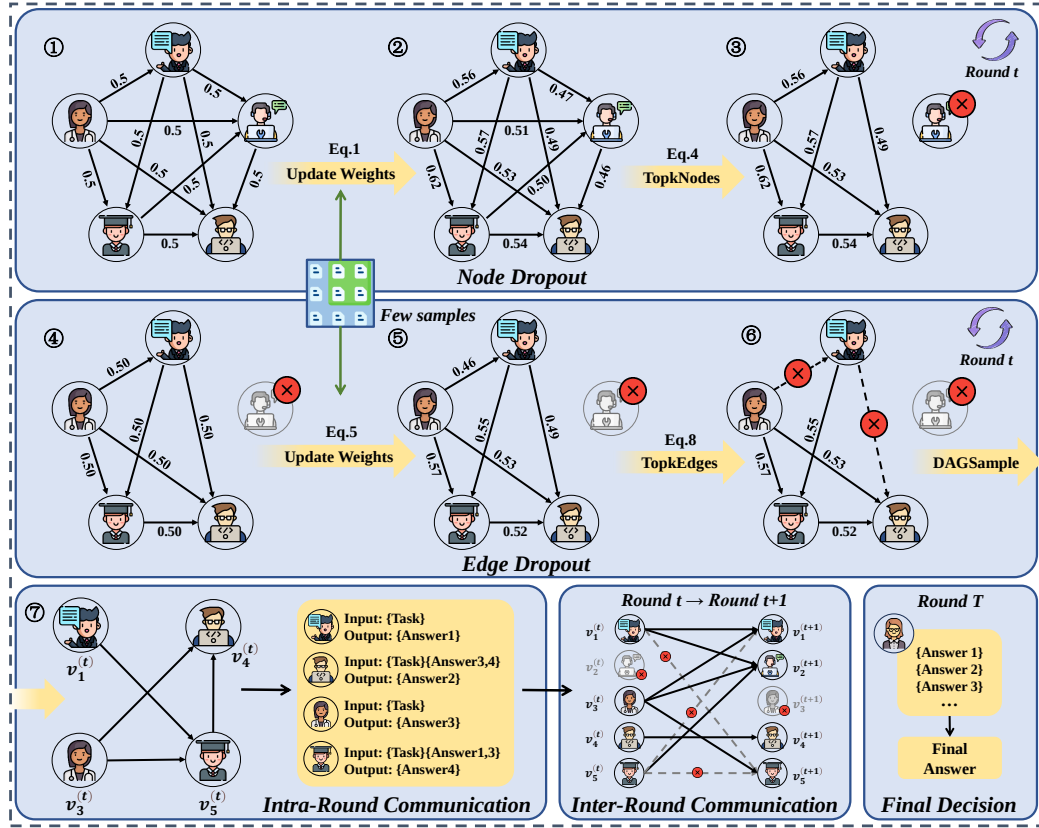


Figure 2: The overall process of AgentDropout. The first and second rows present Node Dropout and Edge Dropout procedures, respectively. The third row illustrates the cooperative reasoning process for both intra- and inter-round communication, as well as the generation of the final answer. For simplicity, the edges of inter-round communication are omitted in the demonstration of step 1~7, but they also participate in the update, dropout, and reasoning processes as shown in the middle block of the third row.

the communication graph to get rid of redundant edges (i.e. $\mathcal{E} \setminus \mathcal{E}'$).

3 AgentDropout

3.1 Overview

Motivation We argue that communication redundancy in MAS primarily arises from two sources: redundant agent roles and redundant information exchange. AgentPrune prunes redundant information exchange but neglects the existence of redundant agent roles and applies the same pruning strategy across all communication rounds, which still leaves redundancy in MAS and may result in performance degradation since different combinations of agent roles might be better suited for different stages of task solving. To this end, we propose a strategy called **AgentDropout** to enhance MAS efficiency and performance. We aim to remove the node that contributes least to the current task from each round’s communication graph $G^{(t)}$ and eliminate both intra- and inter-round edges from the global graph $\mathcal{G}$ that have a minimal positive impact on the task solution.

Preparation We first convert the communication graph into a weighted graph $\tilde{\mathcal{G}}$, where each edge in the graph is a trainable continuous weight in the range of 0 to 1. Let the adjacency matrix set of $\tilde{\mathcal{G}}$ be $\tilde{\mathcal{A}} = \tilde{\mathcal{A}}_{\text{intra}} \cup \tilde{\mathcal{A}}_{\text{inter}}$, where $\tilde{\mathcal{A}}_{\text{intra}} = \bigcup_t \tilde{\mathcal{A}}_{\text{intra}}^{(t)}$ is the subset containing intra-round adjacency matrices, $\tilde{\mathcal{A}}_{\text{intra}}^{(t)} \in [0, 1]^{N \times N}$ is the intra-round adjacency matrix of the round t communication graph and $\tilde{\mathcal{A}}_{\text{inter}} = \bigcup_t \tilde{\mathcal{A}}_{\text{inter}}^{(t)}$ is the subset containing inter-round adjacency matrices, $\tilde{\mathcal{A}}_{\text{inter}}^{(t)} \in [0, 1]^{N \times N}$ is the inter-round adjacency matrix between round $t - 1$ and t . The actual communication graph used during inference $\mathcal{G}$ is obtained by algorithm DAGSample, where each edge is sampled with its own weight as probability, and the output graph is guaranteed to be a directed acyclic graph (DAG). AgentDropout consists of two steps to optimize the graph: **Node Dropout** and **Edge Dropout**.

3.2 Node Dropout

In human teams tackling complex tasks, varying the roles involved in discussions across different rounds can enhance the team’s performance by fo-

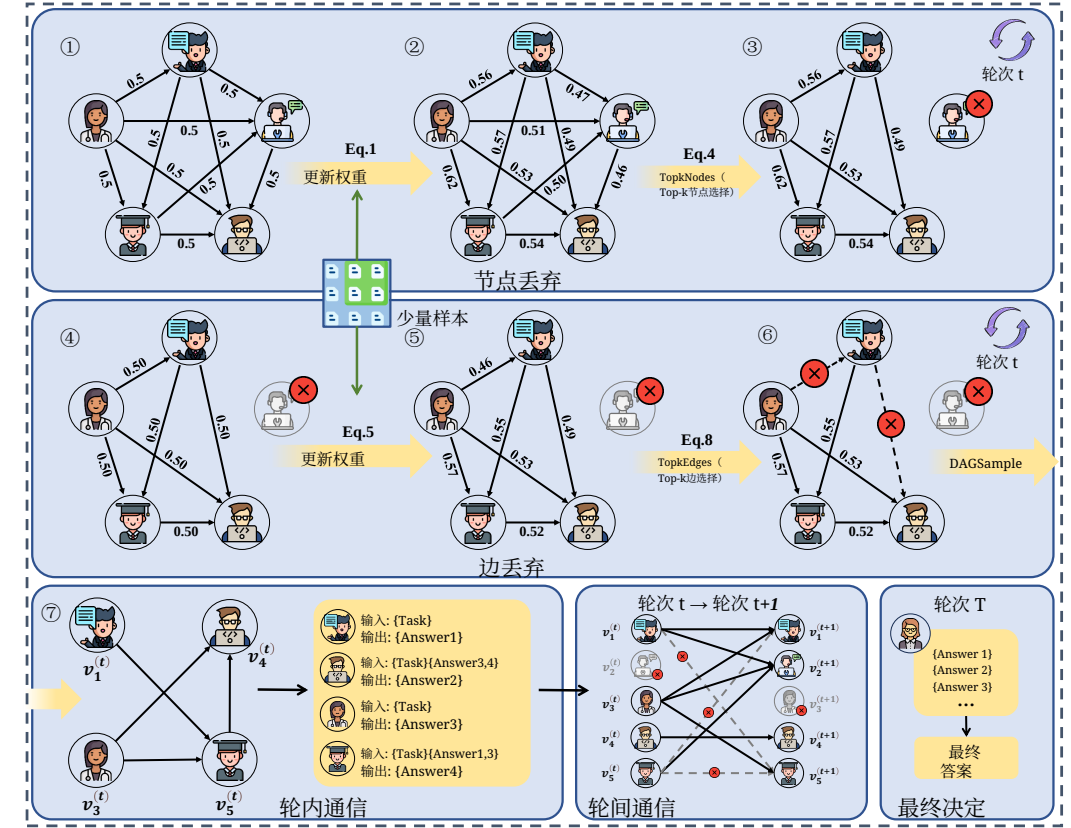


图2: AgentDropout的整体流程。第一行和第二行分别展示了节点丢弃和边丢弃过程。第三行展示了轮内和轮间通信的协作推理过程，以及最终答案的生成。为简洁起见，在步骤 1~7 的演示中，轮间通信的边被省略，但它们同样参与第三行中间部分所示的更新、丢弃和推理过程。

以消除冗余边（即 $\mathcal{E} \setminus \mathcal{E}'$ ）。

3 AgentDropout

3.1 概述

动机 我们认为，多智能体系统中的通信冗余主要源于两个方面：冗余的智能体角色和冗余的信息交换。AgentPrune 修剪了冗余的信息交换，但忽视了冗余智能体角色的存在，并且在所有通信轮次中应用相同的修剪策略，这仍然会在多智能体系统中留下冗余，并可能导致性能下降，因为不同的智能体角色组合可能更适合任务求解的不同阶段。为此，我们提出了一种名为 **AgentDropout** 的策略，以提升多智能体系统的效率和性能。我们的目标是从每轮通信图 $G^{(t)}$ 中移除对当前任务贡献最小的节点，并从全局图 $\mathcal{G}$ 中消除那些对任务求解具有最小正向影响的轮内和轮间边。

准备 我们首先将通信图转换为加权图 $\mathcal{G}$ ，其中图中的每条边都是取值在 0 到 1 范围内的可训练连续权重。设邻接矩阵集合为 $\mathcal{G}$ ，其中 $\tilde{\mathcal{A}} = \tilde{\mathcal{A}}_{\text{intra}} \cup \tilde{\mathcal{A}}_{\text{inter}}$ ， $\tilde{\mathcal{A}}_{\text{intra}} = \bigcup_t \tilde{\mathcal{A}}_{\text{intra}}^{(t)}$ intra 是包含轮内邻接矩阵的子集， $\tilde{\mathcal{A}}_{\text{intra}}^{(t)} \in [0, 1]^{N \times N}$ 是轮次 t 通信图的轮内邻接矩阵， $\tilde{\mathcal{A}}_{\text{inter}} = \bigcup_t \tilde{\mathcal{A}}_{\text{inter}}^{(t)}$ inter 是包含轮间邻接矩阵的子集， $\tilde{\mathcal{A}}_{\text{inter}}^{(t)} \in [0, 1]^{N \times N}$ 是轮次 $t - 1$ 与 t 之间的轮间邻接矩阵。在推理 $\mathcal{G}$ 过程中实际使用的通信图由算法 DAGSample 获得，其中每条边以其自身的权重作为概率进行采样，并且输出的图保证为有向无环图 (DAG)。AgentDropout 包含两个步骤以优化图：节点丢弃和边丢弃。

3.2 节点丢弃

在人类团队处理复杂任务时，在不同轮次中调整参与讨论的角色，能够通过 fo-

cusing on the most relevant contributors at each stage. Therefore, we first remove nodes from the communication graph that have less contribution to each round of communication. As shown in the first row in Figure 2, we initialize the intra- and inter-round adjacency matrices $\tilde{\mathcal{A}}_{\text{intra}} = \left\{ \tilde{\mathbf{A}}_{\text{intra}}^{(t)} \right\}_t$ and $\tilde{\mathcal{A}}_{\text{inter}} = \left\{ \tilde{\mathbf{A}}_{\text{inter}}^{(t)} \right\}_t$ by setting all the edge weights to 0.5.¹ We then train $\tilde{\mathcal{A}}_{\text{intra}}$ to get an optimized weighted communication graph, which is prepared for Node Dropout. The optimization objective is simply high task performance, formulated as:

$$\arg \max_{\tilde{\mathcal{A}}_{\text{intra}}} \mathbb{E}_{\mathcal{G} \sim \mathbb{G}} [\mu(\mathcal{G})] \quad (1)$$

where $\mathbb{G}$ represents the feasible domain of graph sampling. Since the utility function $\mu(\cdot)$ often depends on external APIs, it is commonly non-differentiable. Therefore, we apply a reinforcement algorithm that employs an unbiased policy gradient estimation, as proposed by Zhuge et al. (2024). This approach uses the probability-weighted average performance of a limited number of samples to approximate the overall performance expectation. We sample M communication graphs upon $\tilde{\mathcal{A}}_{\text{intra}}, \tilde{\mathcal{A}}_{\text{inter}}$ using DAGSample and run test instances on each graph $\mathcal{G}_m$ to obtain the performance score $\mu(\mathcal{G}_m)$:

$$\begin{aligned} \nabla_{\tilde{\mathcal{A}}_{\text{intra}}} \mathbb{E}_{\mathcal{G} \sim \mathbb{G}} [\mu(\mathcal{G})] \\ \approx \frac{1}{M} \sum_{m=1}^M \mu(\mathcal{G}_m) \nabla_{\tilde{\mathcal{A}}_{\text{intra}}} \log(p_{\tilde{\mathcal{A}}_{\text{intra}}}(\mathcal{G}_m)) \end{aligned} \quad (2)$$

where $p_{\tilde{\mathcal{A}}_{\text{intra}}}(\mathcal{G}_m)$ represents the probability of sampling the communication graph $\mathcal{G}_m = (\mathcal{F}_m, \mathcal{E}_m, \mathcal{V}_m)$ given the current weighted intra-round adjacency matrices $\tilde{\mathcal{A}}_{\text{intra}}$:

$$p_{\tilde{\mathcal{A}}_{\text{intra}}}(\mathcal{G}_m) = \prod_t \prod_{(v_i, v_j) \in \mathcal{E}_{m, \text{intra}}^{(t)}} \tilde{\mathbf{A}}_{\text{intra}}^{(t)}[i, j] \quad (3)$$

With the gradient calculated, we update $\tilde{\mathcal{A}}_{\text{intra}}$ using the gradient ascent method.

After the graph optimization, we select the nodes with the smallest weighted in- and out-degree in each intra-round graph and remove these nodes with all the edges connected to them:

$$\mathcal{V}_{\Delta} = \bigcup_t \left(\mathcal{V}^{(t)} \setminus \text{TopkNodes}(\tilde{\mathbf{A}}_{\text{inter}}^{(t)}, 1 - \alpha) \right) \quad (4)$$

¹In detail, in the intra-round adjacency matrices, the diagonal elements are 0, and the other elements are 0.5. The inter-round adjacency matrices are upper triangular matrices with diagonal elements as 0 and others as 0.5.

where $\text{TopkNodes}(\cdot)$ is a function returning a set of k nodes with the largest sum of in-degree and out-degree according to the adjacency matrix, and α is the node dropout rate. $\tilde{\mathcal{A}}_{\text{intra}}$ and $\tilde{\mathcal{A}}_{\text{inter}}$ is also modified simultaneously by removing all rows and columns associated with the removed nodes.

3.3 Edge Dropout

Promoting more focused and efficient interactions between roles also improves overall teamwork effectiveness of task completion. Thus, we delete the intra- and inter-round edges in the communication graph that are less important to enhance communication efficiency. As shown in the second row in Figure 2, we reinitialize the intra- and inter-round adjacency matrices $\tilde{\mathbf{A}}_{\text{intra}}^{(t)}, \tilde{\mathbf{A}}_{\text{inter}}^{(t)}$ after Node Dropout and train **both** of them from scratch. Differently, we aim at both high performance and token efficiency this time, formulated as:

$$\begin{aligned} \arg \max_{\tilde{\mathcal{A}}_{\text{intra}}, \tilde{\mathcal{A}}_{\text{inter}}} \mathbb{E}_{\mathcal{G} \sim \mathbb{G}_{\text{Node}}} [\mu(\mathcal{G})] \\ - \left[\sum_{t=1}^T \text{rank}(\tilde{\mathbf{A}}_{\text{intra}}^{(t)}) + \sum_{t=2}^T \text{rank}(\tilde{\mathbf{A}}_{\text{inter}}^{(t)}) \right] \end{aligned} \quad (5)$$

where $\mathbb{G}_{\text{Node}}$ represents the feasible domain of graph sampling after Node Dropout. We also use the weighted average performance of the samples to approximate the performance expectation as in Equation (2). However, different from Equation (2), we calculate the probability and gradient for both $\tilde{\mathcal{A}}_{\text{intra}}$ and $\tilde{\mathcal{A}}_{\text{inter}}$ here:

$$\begin{aligned} p_{\tilde{\mathcal{A}}_{\text{intra}}, \tilde{\mathcal{A}}_{\text{inter}}}(\mathcal{G}_m) = \prod_{t=1}^T \prod_{(v_i, v_j) \in \mathcal{E}_{m, \text{intra}}^{(t)}} \tilde{\mathbf{A}}_{\text{intra}}^{(t)}[i, j] \\ \cdot \prod_{t=2}^T \prod_{(v_i, v_j) \in \mathcal{E}_{m, \text{inter}}^{(t)}} \tilde{\mathbf{A}}_{\text{inter}}^{(t)}[i, j] \end{aligned} \quad (6)$$

The second term here represents low-rank sparsity, which serves as a proxy for communication efficiency. Since low-rank optimization is an NP-hard problem, we replace the rank function with the nuclear norm following Zhang et al. (2025):

$$\arg \min_{\tilde{\mathcal{A}}_{\text{intra}}, \tilde{\mathcal{A}}_{\text{inter}}} \sum_{t=1}^T \|\tilde{\mathbf{A}}_{\text{intra}}^{(t)}\|_* + \sum_{t=2}^T \|\tilde{\mathbf{A}}_{\text{inter}}^{(t)}\|_* \quad (7)$$

Then in each intra- and inter-round adjacency matrix, we select the edges corresponding to the largest elements with a ratio of $1 - \alpha$ to keep and delete the rest of them:

$$\begin{aligned} \mathcal{E}_{\Delta} = \bigcup_t \left(\mathcal{E}_{\text{intra}}^{(t)} \setminus \text{TopkEdges}(\tilde{\mathbf{A}}_{\text{intra}}^{(t)}, 1 - \beta) \right) \\ \cup \bigcup_t \left(\mathcal{E}_{\text{inter}}^{(t)} \setminus \text{TopkEdges}(\tilde{\mathbf{A}}_{\text{inter}}^{(t)}, 1 - \beta) \right) \end{aligned} \quad (8)$$

专注于每个阶段最相关的贡献者。因此，我们首先从通信图中移除对每轮通信贡献较小的节点。如图2第一行所示，我们将所有边权重设置为0.5，以初始化轮内和轮间邻接矩阵 $\tilde{\mathcal{A}}_{\text{intra}} = \left\{ \tilde{\mathbf{A}}_{\text{intra}}^{(t)} \right\}_t$ 和 $\tilde{\mathcal{A}}_{\text{inter}} = \left\{ \tilde{\mathbf{A}}_{\text{inter}}^{(t)} \right\}_t$ 。¹ 随后，我们训练 $\tilde{\mathcal{A}}_{\text{intra}}$ 以获得优化后的加权通信图，该图为节点丢弃做好准备。优化目标简单来说就是高任务性能，其形式化为：

$$\arg \max_{\tilde{\mathcal{A}}_{\text{intra}}} \mathbb{E}_{\mathcal{G} \sim \mathbb{G}} [\mu(\mathcal{G})] \quad (1)$$

其中 $\mathbb{G}$ 表示图采样的可行域。由于效用函数 $\mu(\cdot)$ 通常依赖于外部API，因此通常不可微。因此，我们采用了Zhuge等人 (2024) 提出的强化算法，该算法使用无偏的策略梯度估计。该方法利用有限数量样本的概率加权平均性能来近似整体性能期望。我们在 $\tilde{\mathcal{A}}_{\text{intra}}, \tilde{\mathcal{A}}_{\text{inter}}$ 上使用DAGSample采样 M 个通信图 $\tilde{\mathcal{G}}_m$ ，并在每个图 $\mathcal{G}_m$ 上运行测试实例，以获得性能分数 $\mu(\mathcal{G}_m)$ ：

$$\begin{aligned} \nabla_{\tilde{\mathcal{A}}_{\text{intra}}} \mathbb{E}_{\mathcal{G} \sim \mathbb{G}} [\mu(\mathcal{G})] \\ \approx \frac{1}{M} \sum_{m=1}^M \mu(\mathcal{G}_m) \nabla_{\tilde{\mathcal{A}}_{\text{intra}}} \log(p_{\tilde{\mathcal{A}}_{\text{intra}}}(\mathcal{G}_m)) \end{aligned} \quad (2)$$

其中 $p_{\tilde{\mathcal{A}}_{\text{intra}}}(\mathcal{G}_m)$ 表示在当前加权 $\tilde{\mathcal{A}}_{\text{intra}}$ 内轮次邻接矩阵 $\tilde{\mathcal{A}}_{\text{intra}}$ 条件下，采样通信图 $\mathcal{G}_m = (\mathcal{F}_m, \mathcal{E}_m, \mathcal{V}_m)$ 的概率：

$$p_{\tilde{\mathcal{A}}_{\text{intra}}}(\mathcal{G}_m) = \prod_t \prod_{(v_i, v_j) \in \mathcal{E}_{m, \text{intra}}^{(t)}} \tilde{\mathbf{A}}_{\text{intra}}^{(t)}[i, j] \quad (3)$$

在计算梯度后，我们使用梯度上升法更新 $\tilde{\mathcal{A}}_{\text{intra}}$ 。

在图优化之后，我们在每个轮内图中选择加权入度和出度最小的节点，并移除这些节点及其连接的所有边：

$$\mathcal{V}_{\Delta} = \bigcup_t \left(\mathcal{V}^{(t)} \setminus \text{TopkNodes}(\tilde{\mathbf{A}}_{\text{inter}}^{(t)}, 1 - \alpha) \right) \quad (4)$$

¹具体而言，在轮内邻接矩阵中，对角线元素为 0，其余元素为 0.5。轮间邻接矩阵为上三角矩阵，其对角线元素为 0，其余元素为 0.5。

其中 $\text{TopkNodes}(\cdot)$ 是一个根据邻接矩阵返回入度和出度之和最大的 k 个节点的集合的函数，而 $1 - \alpha$ 是节点丢弃率。 $\tilde{\mathcal{A}}_{\text{intra}}$ 和 $\tilde{\mathcal{A}}_{\text{inter}}$ 也会通过移除与已删除节点相关的所有行和列而同时被修改。

3.3 边丢弃

促进角色之间更专注和高效的互动也能提升任务完成的整体团队协作效果。因此，我们删除通信图中重要性较低的轮内和轮间边，以提升通信效率。如图2第二行所示，在节点丢弃后，我们重新初始化轮内和轮间邻接矩阵 $\tilde{\mathbf{A}}_{\text{intra}}^{(t)}$ 和 $\tilde{\mathbf{A}}_{\text{inter}}^{(t)}$ ，并从零开始训练它们。不同之处在于，此次我们同时追求高性能和令牌效率，其形式化为：

$$\begin{aligned} \arg \max_{\tilde{\mathcal{A}}_{\text{intra}}, \tilde{\mathcal{A}}_{\text{inter}}} \mathbb{E}_{\mathcal{G} \sim \mathbb{G}_{\text{Node}}} [\mu(\mathcal{G})] \\ - \left[\sum_{t=1}^T \text{rank}(\tilde{\mathbf{A}}_{\text{intra}}^{(t)}) + \sum_{t=2}^T \text{rank}(\tilde{\mathbf{A}}_{\text{inter}}^{(t)}) \right] \end{aligned} \quad (5)$$

其中 $\mathbb{G}_{\text{Node}}$ 表示节点丢弃后图采样的可行域。我们同样使用样本的加权平均性能来近似性能期望，如式 (2) 所示。然而，与式 (2) 不同，在此我们对 $\tilde{\mathcal{A}}_{\text{intra}}$ 和 $\tilde{\mathcal{A}}_{\text{inter}}$ 分别计算概率和梯度：

$$\begin{aligned} p_{\tilde{\mathcal{A}}_{\text{intra}}, \tilde{\mathcal{A}}_{\text{inter}}}(\mathcal{G}_m) = \prod_{t=1}^T \prod_{(v_i, v_j) \in \mathcal{E}_{m, \text{intra}}^{(t)}} \tilde{\mathbf{A}}_{\text{intra}}^{(t)}[i, j] \\ \cdot \prod_{t=2}^T \prod_{(v_i, v_j) \in \mathcal{E}_{m, \text{inter}}^{(t)}} \tilde{\mathbf{A}}_{\text{inter}}^{(t)}[i, j] \end{aligned} \quad (6)$$

此处的第二项代表低秩稀疏性，其作为通信效率的代理指标。由于低秩优化是一个 NP 难问题，我们遵循Zhang等 (2025) 的方法，用核范数替代秩函数：

$$\arg \min_{\tilde{\mathcal{A}}_{\text{intra}}, \tilde{\mathcal{A}}_{\text{inter}}} \sum_{t=1}^T \|\tilde{\mathbf{A}}_{\text{intra}}^{(t)}\|_* + \sum_{t=2}^T \|\tilde{\mathbf{A}}_{\text{inter}}^{(t)}\|_* \quad (7)$$

然后，在每轮轮内和轮间邻接矩阵中，我们以 $1 - \alpha$ 的比例保留对应于最大元素的边，并删除其余的边：

$$\begin{aligned} \mathcal{E}_{\Delta} = \bigcup_t \left(\mathcal{E}_{\text{intra}}^{(t)} \setminus \text{TopkEdges}(\tilde{\mathbf{A}}_{\text{intra}}^{(t)}, 1 - \beta) \right) \\ \cup \bigcup_t \left(\mathcal{E}_{\text{inter}}^{(t)} \setminus \text{TopkEdges}(\tilde{\mathbf{A}}_{\text{inter}}^{(t)}, 1 - \beta) \right) \end{aligned} \quad (8)$$

Method	Edge DR.	Node DR.	MMLU	GSM8K	AQuA	MultiArith	SVAMP	HumanEval	Avg.
Base model: Llama3-8B-Instruct									
Vanilla	✗	✗	53.59	70.23	41.67	91.11	75.00	53.33	64.16
CoT	✗	✗	56.86 _{↑3.27}	70.47 _{↑0.24}	43.75 _{↑2.08}	92.25 _{↑1.14}	76.17 _{↑1.67}	54.17 _{↑0.84}	65.61 _{↑1.45}
MAS _{round=1}	✗	✗	56.21 _{↑2.62}	69.30 _{↓0.97}	45.29 _{↑3.62}	91.67 _{↑0.55}	76.67 _{↑1.67}	48.33 _{↓5.00}	64.58 _{↑0.42}
MAS _{round=T}	✗	✗	60.13 _{↑6.54}	71.48 _{↑1.25}	45.41 _{↑3.75}	90.56 _{↓0.55}	77.56 _{↑2.56}	49.17 _{↓4.16}	65.72 _{↑1.56}
AgentPrune	✓	✗	60.78 _{↑7.19}	71.02 _{↑0.79}	47.22 _{↑5.55}	90.00 _{↓1.11}	78.34 _{↑3.34}	51.67 _{↓1.66}	66.51 _{↑2.35}
AgentDropout	✓	✓	62.75 _{↑9.16}	73.13 _{↑2.90}	47.78 _{↑6.11}	92.59 _{↑1.48}	80.11 _{↑5.11}	55.84 _{↑2.51}	68.70 _{↑4.54}
Base model: Qwen2.5-72B-Instruct									
Vanilla	✗	✗	82.35	91.02	83.75	98.89	92.67	85.28	88.99
CoT	✗	✗	83.66 _{↑1.31}	92.19 _{↑1.17}	84.58 _{↑0.83}	100 _{↑1.11}	93.35 _{↑0.68}	86.67 _{↑1.39}	90.08 _{↑1.09}
MAS _{round=1}	✗	✗	82.35 _{↑0.00}	93.52 _{↑2.50}	84.58 _{↑0.83}	100 _{↑1.11}	92.36 _{↓0.31}	84.17 _{↓1.11}	89.50 _{↑0.51}
MAS _{round=T}	✗	✗	84.31 _{↑1.96}	93.28 _{↑2.26}	85.83 _{↑2.08}	100 _{↑1.11}	94.07 _{↑1.40}	87.08 _{↑1.80}	90.76 _{↑1.77}
AgentPrune	✓	✗	83.66 _{↑1.31}	93.67 _{↑2.65}	87.08 _{↑3.33}	99.44 _{↑0.56}	94.33 _{↑1.66}	86.67 _{↑1.39}	90.81 _{↑1.82}
AgentDropout	✓	✓	84.97 _{↑2.62}	93.75 _{↑2.73}	87.50 _{↑3.75}	100 _{↑1.11}	95.34 _{↑2.67}	87.92 _{↑2.64}	91.58 _{↑2.59}
Base model: Deepseek-V3-671B-Instruct									
Vanilla	✗	✗	84.97	94.68	84.58	100	93.67	88.43	91.06
CoT	✗	✗	84.31 _{↓0.66}	95.15 _{↑0.47}	85.42 _{↑0.84}	100 _{↑0.00}	93.94 _{↑0.27}	89.26 _{↑0.83}	91.34 _{↑0.28}
MAS _{round=1}	✗	✗	89.98 _{↑5.01}	95.54 _{↑0.86}	86.67 _{↑2.19}	100 _{↑0.00}	93.50 _{↓0.17}	89.17 _{↑0.74}	92.63 _{↑1.57}
MAS _{round=T}	✗	✗	89.54 _{↑4.57}	95.49 _{↑0.81}	87.50 _{↑2.92}	100 _{↑0.00}	94.33 _{↑0.66}	89.26 _{↑0.83}	92.69 _{↑1.63}
AgentPrune	✓	✗	90.20 _{↑5.23}	95.49 _{↑0.81}	87.92 _{↑3.34}	100 _{↑0.00}	95.00 _{↑1.33}	90.91 _{↑2.47}	93.25 _{↑2.19}
AgentDropout	✓	✓	90.85 _{↑5.88}	95.63 _{↑0.95}	88.33 _{↑3.75}	100 _{↑0.00}	95.79 _{↑2.12}	91.74 _{↑3.31}	93.72 _{↑2.66}

Table 1: Performance comparison between AgentDropout and other baseline reasoning techniques. **Edge DR.** and **Node DR.** represent the Edge Dropout and Node Dropout methods, respectively.

where $\text{TopkEdges}(\cdot)$ is a function returning a set of edges corresponding to top- k elements in the adjacency matrix, and β is the edge dropout rate.

As shown in the third row in Figure 2, the final communication graph $\hat{\mathcal{G}}$ is sampled from the optimized $\tilde{\mathcal{A}}_{\text{intra}}, \tilde{\mathcal{A}}_{\text{inter}}$ using DAGSample. The agents then engage in multi-round communication based on the topology defined in $\hat{\mathcal{G}}$, and their individual responses are then aggregated to generate the final answer. More detailed information on the optimization procedure is presented Appendix A.1.

3.4 Discussion

Previous studies (You et al., 2020; Zhang et al., 2024a) have demonstrated that high-quality topological graphs can be trained using a limited number of samples. Therefore, in our approach, we employ only a small number of samples to update the parameters of the adjacency matrices for the communication graph, subsequently applying these matrices during two dropout processes. The number of these samples is merely in the order of tens, extracted from either the train or the validation set of the dataset, thereby maintaining the training cost and token consumption at relatively low levels.

4 Experiments

4.1 Experimental Setup

Models and Benchmarks We conduct experiments on MAS based on models of varying sizes, namely Meta-Llama3-8B-Instruct (AI@Meta,

2024), Qwen2.5-72B-Instruct (Yang et al., 2024), and Deepseek-V3-671B-Instruct (Liu et al., 2024a). We evaluate the general reasoning capabilities of our method using the MMLU (Hendrycks et al., 2021) dataset; the mathematical reasoning abilities using the GSM8k (Cobbe et al., 2021), MultiArith (Roy and Roth, 2015), AQuA (Patel et al., 2021), and SVAMP (Ling et al., 2017) datasets; and the code generation capabilities using the HumanEval (Chen et al., 2021) dataset.

Baselines For tasks performed by a single agent, we compare with the direct reasoning method, known as Vanilla, and the Chain-of-Thought (CoT, Wei et al., 2022) approach. In the context of multi-agent collaboration, for methods that only communicate within intra-round, we compare the single-round dialogue MAS, denoted as MAS_{round=1}. For methods that communicate both within intra-round and across inter-round, we compare MAS_{round=T} and AgentPrune (Zhang et al., 2025), where round represents the number of inter-round communication rounds among agents. In particular, both AgentPrune and AgentDropout are optimized based on MAS_{round=T}, allowing for a more intuitive comparison of these methods.

Implementation Details For experiments with Llama3 and Qwen2.5, we perform inference using vllm on Nvidia A800 GPUs. For Deepseek-V3, we utilize the Deepseek API to evaluate each method. We set the temperature to 1, the sample times M

方法	边缘动态分辨率。节点丢弃率。		MMLU	GSM8k	AQuA	MultiArith	SVAMP	HumanEval	Avg.
基础模型: Llama3-8B-Instruct									
原版	✗	✗	53.59	70.23	41.67	91.11	75.00	53.33	64.16
CoT	✗	✗	56.86 _{↑3.27}	70.47 _{↑0.24}	43.75 _{↑2.08}	92.25 _{↑1.14}	76.17 _{↑1.67}	54.17 _{↑0.84}	65.61 _{↑1.45}
多智能体系统 _{round=1}	✗	✗	56.21 _{↑2.62}	69.30 _{↓0.97}	45.29 _{↑3.62}	91.67 _{↑0.55}	76.67 _{↑1.67}	48.33 _{↓5.00}	64.58 _{↑0.42}
多智能体系统 _{round=T}	✗	✗	60.13 _{↑6.54}	71.48 _{↑1.25}	45.41 _{↑3.75}	90.56 _{↓0.55}	77.56 _{↑2.56}	49.17 _{↓4.16}	65.72 _{↑1.56}
AgentPrune	✓	✗	60.78 _{↑7.19}	71.02 _{↑0.79}	47.22 _{↑5.55}	90.00 _{↓1.11}	78.34 _{↑3.34}	51.67 _{↓1.66}	66.51 _{↑2.35}
AgentDropout	✓	✓	62.75 _{↑9.16}	73.13 _{↑2.90}	47.78 _{↑6.11}	92.59 _{↑1.48}	80.11 _{↑5.11}	55.84 _{↑2.51}	68.70 _{↑4.54}
基础模型: Qwen2.5-72B-Instruct									
原版	✗	✗	82.35	91.02	83.75	98.89	92.67	85.28	88.99
CoT	✗	✗	83.66 _{↑1.31}	92.19 _{↑1.17}	84.58 _{↑0.83}	100 _{↑1.11}	93.35 _{↑0.68}	86.67 _{↑1.39}	90.08 _{↑1.09}
多智能体系统 _{round=1}	✗	✗	82.35 _{↑0.00}	93.52 _{↑2.50}	84.58 _{↑0.83}	100 _{↑1.11}	92.36 _{↓0.31}	84.17 _{↓1.11}	89.50 _{↑0.51}
多智能体系统 _{round=T}	✗	✗	84.31 _{↑1.96}	93.28 _{↑2.26}	85.83 _{↑2.08}	100 _{↑1.11}	94.07 _{↑1.40}	87.08 _{↑1.80}	90.76 _{↑1.77}
AgentPrune	✓	✗	83.66 _{↑1.31}	93.67 _{↑2.65}	87.08 _{↑3.33}	99.44 _{↑0.56}	94.33 _{↑1.66}	86.67 _{↑1.39}	90.81 _{↑1.82}
AgentDropout	✓	✓	84.97 _{↑2.62}	93.75 _{↑2.73}	87.50 _{↑3.75}	100 _{↑1.11}	95.34 _{↑2.67}	87.92 _{↑2.64}	91.58 _{↑2.59}
基础模型: Deepseek-V3-671B-Instruct									
原版	✗	✗	84.97	94.68	84.58	100	93.67	88.43	91.06
CoT	✗	✗	84.31 _{↓0.66}	95.15 _{↑0.47}	85.42 _{↑0.84}	100 _{↑0.00}	93.94 _{↑0.27}	89.26 _{↑0.83}	91.34 _{↑0.28}
多智能体系统 _{round=1}	✗	✗	89.98 _{↑5.01}	95.54 _{↑0.86}	86.67 _{↑2.19}	100 _{↑0.00}	93.50 _{↓0.17}	89.17 _{↑0.74}	92.63 _{↑1.57}
多智能体系统 _{round=T}	✗	✗	89.54 _{↑4.57}	95.49 _{↑0.81}	87.50 _{↑2.92}	100 _{↑0.00}	94.33 _{↑0.66}	89.26 _{↑0.83}	92.69 _{↑1.63}
AgentPrune	✓	✗	90.20 _{↑5.23}	95.49 _{↑0.81}	87.92 _{↑3.34}	100 _{↑0.00}	95.00 _{↑1.33}	90.91 _{↑2.47}	93.25 _{↑2.19}
AgentDropout	✓	✓	90.85 _{↑5.88}	95.63 _{↑0.95}	88.33 _{↑3.75}	100 _{↑0.00}	95.79 _{↑2.12}	91.74 _{↑3.31}	93.72 _{↑2.66}

表1: 按 AgentDropout与其他基线推理技术的能比较。Edge DR.和Node D 代表 边丢弃和节点丢弃方法， 分别。

其中 TopkEdges (Top-k边选择) ($\cdot$) 是一个函数，返回邻接矩阵中对应于前 k 个元素的边集合，而 β 是边Dropout率。

如图2第三行所示，最终`通信图  $\mathcal{G}$  使用 DAGSample从优化后的`mized $\mathcal{A}_{\text{intra}}, \mathcal{A}_{\text{inter}}$ 中采样得到。然后，智能体基于 $\mathcal{G}$ 中定义的拓扑结构进行多轮通信，并将它们的各个响应进行聚合，以生成最终答案。有关优化过程的更详细信息见附录A.1。

3.4 讨论

先前的研究 (You et al., 2020; Zhang等人, 2024a) 已经证明，使用有限数量的样本可以训练出高质量的拓扑图。因此，在我们的方法中，我们仅使用少量样本来更新通信图邻接矩阵的参数，随后在两个丢弃过程中应用这些矩阵。这些样本的数量仅为数十个，从数据集的训练集或验证集中提取，从而将训练成本和令牌消耗保持在相对较低的水平。

4 实验

4.1 实验设置

模型与基准 我们基于不同规模的模型对多智能体系统 (MAS) 进行实验，即 Meta-Llama3-8B-Instruct (AI@Meta,

2024年)、Qwen2.5-72B-Instruct (Yang 等, 2024年) 和 Deepseek-V3-671B-Instruct (Liu 等, 2024a)。我们使用 MMLU (Hendrycks 等, 2021年) 数据集评估我们方法的通用推理能力；使用 GSM8k (Cobbe 等, 2021年)、MultiArith (Roy 和 Roth, 2015年)、AQuA (Patel 等, 2021年) 和 SVAMP (Ling 等, 2017年) 数据集评估数学推理能力；使用 HumanEval (Chen 等, 2021年) 数据集评估代码生成能力。

基线 对于由单智能体执行的任务，我们与直接推理方法（即原版方法）以及思维链（CoT, Wei et al., 2022）方法进行比较。在多智能体协作的背景下，对于仅在轮内进行通信的方法，我们比较单轮对话多智能体系统，记为MAS_{round=1o}。对于既在轮内又在轮间进行通信的方法，我们比较MAS_{round=T}和 AgentPrune (Zhang等人, 2025年)，其中 round表示智能体之间的轮间通信轮次。特别地，AgentPrune和AgentDropout都是基于 MAS_{round=T}优化的，从而可以更加直观地比较这些方法。

实现细节 对于 Llama3 和 Qwen2.5 的实验，我们在英伟达 A800 GPU 上使用 vllm 进行推理。对于 Deepseek-V3，我们利用 Deepseek API 来评估每种方法。我们将温度参数设置为 1，采样次数 M

Method	MMLU		GSM8K		AQuA		MultiArith		SVAMP		HumanEval		Avg.	
	Ptok.	Ctok.	Ptok.	Ctok.	Ptok.	Ctok.	Ptok.	Ctok.	Ptok.	Ctok.	Ptok.	Ctok.	Ptok.	Ctok.
Vanilla	99K	44K	142K	209K	32K	89K	16K	15k	27K	19K	91K	25K	68K	67K
CoT	129K	70K	154K	337K	34K	105K	18K	23k	30K	50K	93K	29K	76K	102K
MAS _{round=1}	1.4M	355K	8.5M	1.9M	1.1M	390K	1.1M	218K	1.9M	402K	581K	134K	2.4M	567K
MAS _{round=T}	1.6M	387K	16M	3.4M	2.4M	745K	2.1M	388K	3.7M	721K	2.6M	492K	4.7M	1.0M
AgentPrune	1.3M	367K	15M	3.6M	2.0M	759K	1.9M	393K	3.4M	714K	1.5M	465K	4.2M	1.0M
AgentDropout	1.1M	333K	12M	2.8M	1.3M	634K	1.4M	312K	2.6M	594K	1.1M	359K	3.3M	839K

Table 2: Token consumption comparison in the Llama model. **Ptok.** denotes the number of prompt tokens for the agents, while **Ctok.** represents the number of completion tokens generated by the agents.

Graph	Method	MMLU	GSM8K	AQuA	MultiArith	SVAMP	HumanEval	Avg.	Ptok.	Ctok.
Layered	MAS	57.52	71.72	47.92	87.26	78.84	52.89	66.13	4.3M	1.1M
	AgentPrune	59.48	71.09	46.67	88.93	78.67	56.20	66.84	3.7M	1.0M
	AgentDropout	61.44	72.27	49.17	90.25	79.75	57.02	68.32	2.8M	797K
Random	MAS	62.75	70.63	46.67	87.22	77.17	50.41	65.81	4.2M	1.0M
	AgentPrune	63.40	69.61	44.17	91.11	78.34	51.24	66.31	3.4M	1.0M
	AgentDropout	64.71	72.03	47.50	91.67	79.50	53.72	68.19	2.7M	834K

Table 3: Performance and average token consumption achieved with different initial communication graph topological structures.

in {10, 20}, the learning rate η to 0.1, and the note and edge dropout rate α and β in {0.1,0.2}, respectively. For all benchmarks, the number of training samples we extracted is 40. For general reasoning tasks and mathematical tasks, we set the number of communication rounds $T = 2$, while for code generation tasks, we configure $T = 4$. To ensure a fair comparison, we directly adopt the agent configuration files from AgentPrune.

4.2 Main Result

AgentDropout outperforms other baselines on multiple benchmarks. As shown in Table 1, our method consistently outperforms both the single-agent method CoT and the state-of-the-art multi-agent communication method AgentPrune across all benchmarks, regardless of the model scale. Specifically, using the Llama3 model, AgentDropout improves accuracy by 2.19 compared to the SOTA method. In experiments with Qwen2.5 and Deepseek-V3, the accuracy improves by 0.77 and 0.47, respectively. These results demonstrate the effectiveness of AgentDropout. A case study of the reasoning results is provided in Appendix A.2.

AgentDropout improves the stability of MAS. In Table 1, it can be observed that when the model capability is relatively weak (e.g., Llama3), optimization methods for single agents, such as CoT, exhibit more significant performance improvements, whereas the performance gains from multi-agent interaction and collaboration are less stable, as seen in MAS and AgentPrune. However, when the model capability is sufficiently strong,

the performance improvements brought by single-agent optimization methods diminish, while multi-agent communication methods achieve more stable enhancements. Notably, for models of different scales, AgentDropout consistently enables stable performance improvements in MAS, demonstrating its model robustness.

AgentDropout reduces the token cost of multi-agent communication and cooperation. As shown in Table 2, compared to the multi-round communication method MAS_{round=T}, AgentDropout significantly reduces token consumption. In contrast to AgentPrune, which primarily reduces prompt token consumption, AgentDropout not only further decreases prompt tokens but also significantly reduces completion tokens, revealing its token efficiency. For more details, please refer to Appendix A.3.

4.3 Analysis

In this part, we conducted experiments using the Llama3 model, while maintaining the same experimental settings as described in Section 4.1.

Structure Robustness To verify the stability of AgentDropout when initialized with different topological structures, we also conducted supplementary experiments on communication graphs initialized with layered and random graphs, which are introduced in Appendix A.4. As shown in Table 3, AgentDropout consistently enhances the performance and reduces the average consumption of prompt tokens and completion tokens of

方法	MMLU		GSM8k		AQuA		MultiArith		SVAMP		HumanEval		Avg.	
	提示词令牌数	上下文令牌数	提示词令牌数	上下文令牌数	提示词令牌数	上下文令牌数	提示词令牌数	上下文令牌数	提示词令牌数	上下文令牌数	提示词令牌数	上下文令牌数	提示词令牌数	上下文令牌数
原版	99K	44K	142K	209K	32K	89K	16K	15k	27K	19K	91K	25K	68K	67K
CoT	129K	70K	154K	337K	34K	105K	18K	23k	30K	50K	93K	29K	76K	102K
多智能体系统 _{round=1}	1.4M	355K	8.5M	1.9M	1.1M	390K	1.1M	218K	1.9M	402K	581K	134K	2.4M	567K
多智能体系统 _{round=T}	1.6M	387K	16M	3.4M	2.4M	745K	2.1M	388K	3.7M	721K	2.6M	492K	4.7M	1.0M
AgentPrune	1.3M	367K	15M	3.6M	2.0M	759K	1.9M	393K	3.4M	714K	1.5M	465K	4.2M	1.0M
AgentDropout	1.1M	333K	12M	2.8M	1.3M	634K	1.4M	312K	2.6M	594K	1.1M	359K	3.3M	839K

表2: Llama 模型中的令牌消耗比较。Ptok. 表示智能体的提示令牌数，而 Ctok. 表示智能体生成的补全令牌数。

Graph	方法	MMLU	GSM8k	AQuA	MultiArith	SVAMP	HumanEval	平均值	Ptok.	Ctok.
分层	MAS	57.52	71.72	47.92	87.26	78.84	52.89	66.13	4.3M	1.1M
	AgentPrune	59.48	71.09	46.67	88.93	78.67	56.20	66.84	3.7M	1.0M
	AgentDropout	61.44	72.27	49.17	90.25	79.75	57.02	68.32	2.8M	797K
随机	MAS	62.75	70.63	46.67	87.22	77.17	50.41	65.81	4.2M	1.0M
	AgentPrune	63.40	69.61	44.17	91.11	78.34	51.24	66.31	3.4M	1.0M
	AgentDropout	64.71	72.03	47.50	91.67	79.50	53.72	68.19	2.7M	834K

表3: 性能及在不同初始通信图拓扑结构下实现的平均令牌消耗

s.

在 {10, 20} 之间，学习率 η 为 0.1，节点和边Dropout率 α 和 β 分别为 {0.1,0.2}。对于所有基准测试，我们提取的训练样本数量为 40。对于通用推理任务和数学任务，我们将通信轮次设置为 $T = 2$ ；而对于代码生成任务，我们配置为 $T = 4$ 。为确保公平比较，我们直接采用 AgentPrune 的智能体配置文件。

4.2 主要结果

AgentDropout在多个基准测试上优于其他基线。如表1所示，无论模型规模如何，我们的方法在所有基准测试中均一致地优于单智能体方法思维链（CoT）以及最先进多智能体通信方法AgentPrune。具体而言，在使用 Llama3模型时，AgentDropout相较于最先进方法（SOTA）将准确率提高了2.19。在 Qwen2.5和Deepseek-V3的实验中，准确率分别提高了0.77和0.47。这些结果证明了 AgentDropout的有效性。附录A.2提供了推理结果的案例研究。

AgentDropout 提升了多智能体系统（MAS）的稳定性。从表1中可以观察到，当模型能力相对较弱（例如 Llama3）时，单智能体的优化方法，如思维链（CoT），表现出更显著的性能提升，而多智能体交互和协作带来的性能增益则不够稳定，如 MAS 和 AgentPrune 所示。然而，当模型能力足够强时，

单智能体优化方法带来的性能提升会减弱，而多智能体通信方法则能实现更稳定的增强。值得注意的是，对于不同规模的模型，AgentDropout 始终能够在 MAS 中实现稳定的性能提升，从而展示了其模型鲁棒性。

AgentDropout降低了多智能体通信与协作的令牌成本。如表2所示，与多轮次通信方法 MAS_{round=T}相比，AgentDropout显著降低了令牌消耗。与主要降低提示令牌消耗的 AgentPrune相比，AgentDropout不仅进一步减少了提示令牌，还显著降低了补全令牌，体现了其令牌效率。更多详情请参阅附录 A.3。

4.3 分析

在本部分，我们使用Llama3模型进行了实验，同时保持与4.1节中所述的相同实验设置。

结构鲁棒性 为了验证AgentDropout在不同拓扑结构初始化时的稳定性，我们还对使用分层图和随机图初始化的通信图进行了补充实验，这些内容在附录A.4中有介绍。如表3所示，AgentDropout在不同图结构上持续提升性能并降低提示令牌和补全令牌的平均消耗

Dropout Rate	MMLU	GSM8K	AQuA	MultiArith	SVAMP	HumanEval	Avg.	Ptok.	Ctok.
0.8	57.52	70.86	46.67	89.44	78.67	52.89	66.01	856K	230K
0.6	60.78	71.09	45.42	89.44	80.37	50.41	66.25	1.3M	434K
0.4	60.13	73.36	44.99	90.00	78.33	52.07	66.48	1.9M	648K
0.2	62.75	73.13	47.78	92.59	80.11	55.84	68.70	3.3M	839K

Table 4: Impact of varying dropout rates on the performance and average token consumption of AgentDropout.

Method	Edge DR.	Node DR.	MMLU	GSM8K	AQuA	MultiArith	SVAMP	HumanEval	Avg.
MAS	✗	✗	60.13	71.48	45.41	90.56	77.56	49.17	65.72
Random Dropout	✗	✓	61.72	70.53	43.75	92.23	79.00	52.89	66.69
	✓	✗	60.13	71.41	46.05	90.27	77.67	51.23	66.13
Learned Dropout	✗	✓	63.76	70.70	46.25	90.28	79.17	50.83	66.83
	✓	✗	61.87	70.63	47.08	90.00	78.03	48.76	66.06
Single Learning	✓	✓	60.78	72.58	46.25	91.67	78.83	48.73	66.47
AgentDropout	✓	✓	62.75	73.13	47.78	92.59	80.11	55.84	68.70

Table 5: Performance achieved with different dropout strategies. For Random Dropout, ✓ denotes the random dropout strategy is applied to the marked dropout step, while the dropout strategy in the other step remains unchanged, as per AgentDropout.

Method	GSM8K	AQuA	MultiArith	SVAMP	Avg.
GSM8K	73.13	46.67	92.22	80.33	73.09
AQuA	73.84	47.78	93.13	78.21	73.24
MultiArith	71.41	47.50	92.59	78.33	72.46
SVAMP	70.86	47.50	91.95	80.11	72.61
AgentPrune	71.02	47.22	90.00	78.34	71.65
AgentDropout	73.13	47.78	92.59	80.11	73.40

Table 6: Performance comparison using different combinations of training and test sets. The row headers indicate the training set used for graph topology learning, and the column headers represent the test set.

MAS across various graph structures, which further demonstrates its robustness.

Effect of Dropout Rate To investigate the impact of different dropout rates on the performance of MAS, we conducted experiments with varying values of α and β , as shown in Table 4. The results demonstrate that higher dropout rates lead to a corresponding decline in the average performance of MAS. However, compared with the SOTA methods, AgentDropout still achieves superior average performance across different dropout rates, which substantiates the effectiveness of dropout. In extreme cases, higher dropout rates can be set to ensure efficiency. Nevertheless, for tasks such as AQuA and HumanEval, MAS exhibits lower sensitivity to dropout rates, suggesting that future work could focus on designing more appropriate MAS frameworks for such scenarios.

Different Dropout Strategies To verify the effectiveness of our dropout strategy, we conducted a series of comparative experiments. First, we performed an ablation study on the overall AgentDropout methodology, comparing it against sce-

narios that use only Node Dropout and only Edge Dropout. Additionally, to rule out the possibility that the performance improvement is simply due to the shortening of the reasoning chain, we conduct an experiment where the proposed dropout mechanism is replaced with a random dropout approach in either the Node Dropout or Edge Dropout stage while keeping all other processes unchanged.

As shown in the ‘‘Learned Dropout’’ block of Table 5, applying either Node Dropout or Edge Dropout alone results in improvements compared to the vanilla MAS, indicating the reasonableness of the dropout approaches. However, the simultaneous application of both Node Dropout and Edge Dropout, referred to as ‘‘AgentDropout’’, yields the optimal performance. In the random dropout experiments, randomly dropping either edges or nodes within the AgentDropout framework leads to performance degradation, further substantiating the effectiveness of our proposed dropout strategy.

Training Strategy To validate the effectiveness of our two-round adjacency matrix learning strategy (one for Node Dropout and one for Edge Dropout), we conducted comparative experiments with single-round learning. Specifically, we trained both intra- and inter-round adjacency matrices simultaneously, and then directly applied them to perform Node and Edge Dropout on MAS. The experimental results are presented in Table 5. The performance of AgentDropout significantly outperforms that of single-round learning. This improvement can be attributed to the fact that after Node Dropout, the communication graph generates new inference chains, necessitating an additional train-

丢弃率	MMLU	GSM8k	AQuA	MultiArith	SVAMP	HumanEval	平均值	推理链长度	上下文词元数
0.8	57.52	70.86	46.67	89.44	78.67	52.89	66.01	856K	230K
0.6	60.78	71.09	45.42	89.44	80.37	50.41	66.25	1.3M	434K
0.4	60.13	73.36	44.99	90.00	78.33	52.07	66.48	1.9M	648K
0.2	62.75	73.13	47.78	92.59	80.11	55.84	68.70	3.3M	839K

表4：不同丢弃率对AgentDropout的性能和平均令牌消耗的影响

方法	边缘丢弃率。	节点丢弃率。	MMLU	GSM8k	AQuA	MultiArith	SVAMP	HumanEval	平均值
MAS	✗	✗	60.13	71.48	45.41	90.56	77.56	49.17	65.72
随机Dropout	✗	✓	61.72	70.53	43.75	92.23	79.00	52.89	66.69
	✓	✗	60.13	71.41	46.05	90.27	77.67	51.23	66.13
学习式丢弃	✗	✓	63.76	70.70	46.25	90.28	79.17	50.83	66.83
	✓	✗	61.87	70.63	47.08	90.00	78.03	48.76	66.06
单步学习	✓	✓	60.78	72.58	46.25	91.67	78.83	48.73	66.47
AgentDropout	✓	✓	62.75	73.13	47.78	92.59	80.11	55.84	68.70

表 5：使用不同Dropout策略所达到的性能。对于随机Dropout，✓ 表示随机丢弃策略应用于标记的Dropout步骤，而另一个步骤中的丢弃策略保持不变，遵循AgentDropout。

方法	GSM8K	AQuA	MultiArith	SVAMP	平均值
GSM8K	73.13	46.67	92.22	80.33	73.09
AQuA	73.84	47.78	93.13	78.21	73.24
MultiArith	71.41	47.50	92.59	78.33	72.46
SVAMP	70.86	47.50	91.95	80.11	72.61
AgentPrune	71.02	47.22	90.00	78.34	71.65
AgentDropout	73.13	47.78	92.59	80.11	73.40

表 6：使用不同训练集和测试集组合的性能比较。行标题表示用于图拓扑学习的训练集，列标题表示测试集。

在多智能体系统（MAS）中，这进一步证明了其鲁棒性。

丢弃率的影响为了探究不同丢弃率对多智能体系统（MAS）性能的影响，我们在表4中展示了 α 和 β 取不同值时的实验结果。结果表明，较高的丢弃率会导致多智能体系统平均性能相应的下降。然而，与最先进（SOTA）方法相比，AgentDropout在不同的丢弃率下仍实现了更优越的平均性能，这证明了丢弃机制的有效性。在极端情况下，可以设置较高的丢弃率以确保效率。尽管如此，对于AQuA和HumanEval等任务，多智能体系统对丢弃率表现出较低的敏感性，这表明未来的工作可以专注于为这些场景设计更合适的多智能体系统框架。

不同的Dropout策略 为了验证我们Dropout策略的有效性，我们进行了一系列对比实验。首先，我们对整个AgentDropout方法进行了消融研究，将其与仅使用节点丢弃

和仅使用边丢弃的场景进行了比较。此外，为了排除性能提升仅仅是由于推理链缩短的可能性，我们进行了一项实验，将所提出的Dropout机制替换为随机丢弃方法，分别应用于节点丢弃或边丢弃阶段，同时保持其他所有流程不变。

如表 5 中“学习式丢弃”模块所示，单独应用节点丢弃或边丢弃均相较于原始多智能体系统有所提升，表明丢弃策略的合理性。然而，同时应用节点丢弃和边丢弃，即所谓的“AgentDropout”，能够获得最优性能。在随机Dropout实验中，在AgentDropout框架内随机丢弃边或节点会导致性能下降，进一步证实了我们所提出的丢弃策略的有效性。

训练策略 为验证我们两阶段邻接矩阵学习策略（一个用于节点丢弃，另一个用于边丢弃）的有效性，我们进行了与单轮学习相比的对比实验。具体而言，我们同时训练了轮内和轮间邻接矩阵，然后直接将其应用于对多智能体系统执行节点和边丢弃。实验结果见表 5。AgentDropout 的性能显著优于单轮学习。这一提升可以归因于，在节点丢弃之后，通信图会生成新的推理链，因此需要额外的训练轮次来有效实施进一步的丢弃。

ing round to effectively implement further dropout.

Domain Transferability Our approach requires some training instances of the current task to conduct communication graph optimization and Node/Edge dropout. However, in the evaluation phase, there may be no training data available that directly corresponds to the current test data. To this point, we conduct a domain robustness test, assessing how well our system works when only roughly similar training data is available. Specifically, we utilize dataset A to optimize the communication topology graph and evaluate the trained MAS on dataset B. We conduct experiments on mathematics-related datasets.

As shown in Table 6. When using other datasets as training samples, AgentDropout still maintains robust performance. When trained on the more challenging AQuA dataset, it achieves an average performance improvement of 1.59 over the SOTA method, while for the simpler MultiArith dataset, the average improvement is 0.81. This demonstrates that the communication subgraphs learned by AgentDropout for tasks of the same category exhibit strong generalizability. Moreover, the effectiveness of AgentDropout is positively correlated with the difficulty of the training samples. Specifically, for tasks of the same type, the more challenging the task, the stronger the generalizability learned by MAS. Therefore, when applying AgentDropout to few-shot tasks, selecting similar datasets for training is recommended, which further validates its domain robustness.

5 Related Work

5.1 LLM-based MAS

MAS based on LLMs has emerged as a crucial tool for tackling complex tasks across various domains. Early research has demonstrated the significant potential of LLM-driven agents in collaboratively solving problems through multi-agent debates (Liang et al., 2024; Du et al., 2024), laying the foundation for several research directions, including role-playing in complex reasoning (Wang et al., 2024c; Chen et al., 2024a), collaborative software development (Ishibashi and Nishimura, 2024; Qian et al., 2024c; Hong et al., 2024), and embodied agent interactions (Mandi et al., 2024; Zhang et al., 2024b; Guo et al., 2024). Recent studies have shown that increasing the number and diversity of agents can enhance the overall performance of MAS (Wang et al., 2025a; junyou li et al., 2024;

Chen et al., 2025). As LLM-based MAS continues to scale in size and complexity, the issues of computational cost and communication efficiency have become more pronounced (Chen et al., 2024b; Li et al., 2024). Recent work has proposed methods to optimize MAS efficiency, such as Optima (Chen et al., 2024c) and AgentPrune (Zhang et al., 2025). In this paper, we address the problem of communication redundancy by applying dropout to nodes and edges in the graph structure of MAS, significantly improving computational cost and communication efficiency in practical applications.

5.2 MAS as Graphs

In recent years, researchers have proposed learning graph connections to enhance the collaborative performance of MAS (Hu et al., 2024; Pesce and Montana, 2023). Prior to the widespread adoption of LLMs, existing methods explored optimal communication graph structures for MAS using techniques such as graph diffusion, weighted GNNs, and transformers (Liu et al., 2022; Pesce and Montana, 2023; Hu et al., 2024). For LLM-based MAS, previous work has also attempted to use graphs to represent interactions between agents. For instance, methods like ChatEval (Chan et al., 2024) and AutoGen (Wu et al., 2023) implicitly incorporate graph structures to model simultaneous conversations, while other approaches such as STOP (Zelikman et al., 2023) and DSPy (Khatab et al., 2024) optimize prompt and reasoning structures. Additionally, methods like MacNet (Qian et al., 2025), GPTSwarm (Zhuge et al., 2024), EvoMAC (Hu et al., 2025) and DyLAN (Liu et al., 2024b) optimize the topology of agents communication.

6 Conclusion

In this paper, inspired by management theories that roles in efficient teams are often dynamically adjusted, we propose AgentDropout, a novel topology optimization method for MAS with domain transferability and structure robustness. AgentDropout dynamically adjusts the participating agents and communication links among agents in each round, allowing for more flexible and adaptive team configurations. Experiments on a series of tasks demonstrate that AgentDropout not only reduces token consumption but also consistently enhances the performance of MAS across various tasks. This provides valuable insights for future exploration of larger-scale multi-agent interactions.

i下一轮以有效实施进一步的淘汰 ut.

领域可迁移性 我们的方法需要当前任务的一些训练实例来执行通信图优化和节点/边随机失活。然而，在评估阶段，可能没有直接对应当前测试数据的可用训练数据。为此，我们进行了一项领域鲁棒性测试，以评估在仅有大致相似的可用训练数据时系统的运行效果。具体而言，我们利用数据集A来优化通信拓扑结构图，并在数据集B上评估训练后的多智能体系统（MAS）。我们在数学相关数据集上进行了实验。

如表6所示，当使用其他数据集作为训练样本时，AgentDropout仍能保持鲁棒的性能。在更具挑战性的AQuA数据集上进行训练时，与最先进方法相比，其平均性能提升了1.59；而在较简单的MultiArith数据集上，平均提升为0.81。这表明，AgentDropout为同类别任务学习到的通信子图具有很强的泛化能力。此外，AgentDropout的有效性同训练样本的难度呈正相关。具体而言，对于同类型任务，任务越具挑战性，多智能体系统学到的泛化能力越强。因此，在将AgentDropout应用于少样本任务时，建议选择相似的数据集进行训练，这进一步验证了其领域鲁棒性。

5 相关工作

5.1 基于大语言模型的多智能体系统

基于大语言模型的多智能体系统已成为解决各个领域复杂任务的重要工具。早期研究已展示了大语言模型驱动的智能体通过多智能体辩论协作解决问题的巨大潜力（Liang等人，2024年；Du等人，2024年），这为多个研究方向奠定了基础，包括复杂推理中的角色扮演（Wang等人，2024c；Chen等人，2024a）、协作软件开发（Ishibashi和Nishimura，2024年；Qian等人，2024c；Hong等人，2024年）以及具身智能体交互（Mandi等人，2024年；Zhang等人，2024b；Guo等人，2024年）。最近的研究表明，增加智能体的数量和多样性可以提升多智能体系统的整体性能（Wang等人，2025a；junyou li等人，2024年；

（Chen等人，2025年）。随着基于大语言模型的多智能体系统在规模和复杂度上不断扩展，计算成本和通信效率问题日益突出（Chen等人，2024b；Li等人，2024年）。近期研究提出了多种优化多智能体系统效率的方法，例如Optima（Chen等人，2024c）和AgentPrune（Zhang等人，2025年）。本文通过在大语言模型多智能体系统的图结构中对节点和边应用dropout技术，解决了通信冗余问题，显著提升了实际应用中的计算成本和通信效率。

5.2 多智能体系统作为图

近年来，研究人员提出了学习图连接的方法，以提升多智能体系统的协作性能（Hu等人，2024年；Pesce和Montana，2023年）。在大语言模型广泛应用之前，现有方法利用图扩散、加权图神经网络和Transformer等技术，为多智能体系统探索最优的通信图结构（Liu等人，2022年；Pesce和Montana，2023年；Hu等人，2024年）。对于基于大语言模型的多智能体系统，先前的研究也尝试使用图来表示智能体之间的交互。例如，ChatEval（Chan等人，2024年）和AutoGen（Wu等人，2023年）等方法隐式地融入图结构以建模同时进行的对话，而STOP（Zelikman等人，2023年）和DSPy（Khatab等人，2024年）等其他方法则优化提示词和推理结构。此外，MacNet（Qian等人，2025年）、GPTSwarm（Zhuge等人，2024年）、EvoMAC（Hu等人，2025年）和DyLAN（Liu等人，2024b）等方法优化智能体通信的拓扑结构。

6 结论

在本文中，受管理理论中高效团队的角色往往需要动态调整这一观点的启发，我们提出了AgentDropout，一种用于多智能体系统（MAS）的新型拓扑优化方法，该方法具备领域迁移能力和结构鲁棒性。AgentDropout动态调整每一轮次中参与任务的智能体以及智能体之间的通信链路，从而实现更加灵活和自适应的团队配置。一系列任务上的实验表明，AgentDropout不仅降低了令牌消耗，还在各种任务中持续提升多智能体系统的性能。这为未来探索更大规模的多智能体交互提供了有价值的见解。

Limitations

The tasks validated in this paper are relatively limited and focus mainly on conventional reasoning tasks, mathematical tasks, and code generation tasks. To better evaluate the generalizability of AgentDropout, additional experiments across a broader and more diverse range of tasks are essential. Furthermore, although AgentDropout demonstrates promising efficiency and performance, the current framework remains constrained by predefined agent roles and prompts. When addressing tasks in unfamiliar domains, the careful design of new initial roles and prompts is still crucial. Future work could also explore integrating methods for automatically designing roles and prompts to enhance flexibility. Additionally, although AgentDropout reduces the token consumption of MAS, it remains relatively high, indicating significant opportunities for further optimization.

References

Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. [Gpt-4 technical report](#). *ArXiv preprint*, abs/2303.08774.

AI@Meta. 2024. [Llama 3 model card](#). *None*.

Chi-Min Chan, Weize Chen, Yusheng Su, Jianxuan Yu, Wei Xue, Shanghang Zhang, Jie Fu, and Zhiyuan Liu. 2024. [Chateval: Towards better LLM-based evaluators through multi-agent debate](#). In *The Twelfth International Conference on Learning Representations*.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sas-try, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. [Evaluating large language models trained on code](#). *ArXiv preprint*, abs/2107.03374.

Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, Yujia Qin, Xin Cong, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. 2024a. [Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors](#). In *The Twelfth International Conference on Learning Representations*.

Weize Chen, Ziming You, Ran Li, Yitong Guan, Chen Qian, Chenyang Zhao, Cheng Yang, Ruobing Xie, Zhiyuan Liu, and Maosong Sun. 2025. [Internet of agents: Weaving a web of heterogeneous agents for collaborative intelligence](#). In *The Thirteenth International Conference on Learning Representations*.

Weize Chen, Chenfei Yuan, Jiarui Yuan, Yusheng Su, Chen Qian, Cheng Yang, Ruobing Xie, Zhiyuan Liu, and Maosong Sun. 2024b. [Beyond natural language: LLMs leveraging alternative formats for enhanced reasoning and communication](#). In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 10626–10641, Miami, Florida, USA. Association for Computational Linguistics.

Weize Chen, Jiarui Yuan, Chen Qian, Cheng Yang, Zhiyuan Liu, and Maosong Sun. 2024c. [Optima: Optimizing effectiveness and efficiency for llm-based multi-agent system](#). *ArXiv preprint*, abs/2410.08115.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. [Training verifiers to solve math word problems](#). *ArXiv preprint*, abs/2110.14168.

Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. 2024. [Improving factuality and reasoning in language models through multiagent debate](#). In *Forty-first International Conference on Machine Learning*.

Shen Gao, Hao Li, Zhengliang Shi, Chengrui Huang, Quan Tu, Zhiliang Tian, Minlie Huang, and Shuo Shang. 2024. [360 {deg} rea: Towards a reusable experience accumulation with 360 {deg} assessment for multi-agent system](#). *ArXiv preprint*, abs/2404.05569.

Xudong Guo, Kaixuan Huang, Jiale Liu, Wenhui Fan, Natalia Vélez, Qingyun Wu, Huazheng Wang, Thomas L Griffiths, and Mengdi Wang. 2024. [Embodied llm agents learn to cooperate in organized teams](#). *ArXiv preprint*, abs/2403.12482.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. [Measuring massive multitask language understanding](#). In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net.

Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang

局限性

本文验证的任务相对有限，主要集中于常规推理任务、数学任务和代码生成任务。为了更好地评估AgentDropout的泛化能力，在更广泛、更多样化的任务上开展额外实验是必不可少的。此外，尽管AgentDropout展现出令人满意的效率与性能，但当前框架仍受限于预定义的智能体角色和提示词。在应对不熟悉领域的任务时，精心设计新的初始角色和提示词仍然是至关重要的。未来的工作也可以探索集成用于自动设计角色和提示词的方法，以增强灵活性。另外，尽管AgentDropout降低了多智能体系统的令牌消耗，但总体水平仍然相对较高，这表明进一步优化存在显著空间。

参考文献

Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023年. GPT-4 技术报告. *ArXiv预印本*, abs/2303.08774.

AI@Meta. 2024年. Llama 3 模型卡. *None*.

Chi-Min Chan, 陈伟泽, 玉生 Su, Jianxuan Yu, Wei Xue, Shanghang Zhang, Jie Fu, and 刘知远. 2024年. Chateval: 通过多智能体辩论提升基于大语言模型的评估器. In *The Twelfth International Conference on Learning Representations*.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sas-try, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, MiraMurati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021年. Evaluat- ing large language models trained on code. *ArXiv预印本*, abs/2107.03374.

陈伟泽, 玉生苏, 左靖伟, 杨成, 源辰飞, Chi-Min Chan, 余海阳, 卢雅溪, Yi-Hsin Hung, 钱晨, 秦钰嘉, 丛信, 谢若冰, 刘知远, 孙茂松, 和 周杰. 2024a. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors. In 第十二届学习表征国际会议.

陈伟泽, Ziming You, Ran Li, Yitong Guan, 钱晨, Chenyang Zhao, 杨成, Ruobing Xie, 刘知远, and 孙茂松. 2025年. Internet of agents: 编织异构智能体网络以实现协作智能. In *The Thirteenth International Conference on Learning Representations*.

Weize Chen, Chenfei Yuan, Jiarui Yuan, 玉生 Su, 钱晨, 杨成, Ruobing Xie, 刘知远, 孙茂松. 2024b. 超越自然语言：大语言模型利用替代格式以增强推理与沟通。载于 计算语言学协会研究结果：2024年自然语言处理实证方法会议，页码 10626–10641，迈阿密，佛罗里达，美国。计算语言学协会。

Weize Chen, Jiarui Yuan, 钱晨, 杨成, 刘知远, 孙茂松. 2024c. Optima: 优化基于大语言模型的多智能体系统的有效性 with 效率。ArXiv预印本，abs/2410.08115。

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, John Schulman. 2021年. 训练验证器以解决数学应用题。ArXiv预印本，abs/2110.14168。

杜一伦、Shuang Li、Antonio Torralba、乔舒亚·B·特宁鲍姆、Igor Mordatch。2024年。通过多智能体辩论提高大语言模型的事实性和推理能力。载于第四十一届机器学习国际会议。

Shen Gao、Hao Li、Zhengliang Shi、Chengrui Huang、Quan Tu、Zhiliang Tian、Minlie Huang、Shuo Shang。2024年。[360 {deg} rea](#)：迈向具有 [360 {deg}](#) 评估的多智能体系统可复用经验积累。ArXiv预印本，abs/2404.05569。

Xudong Guo、Kaixuan Huang、Jiale Liu、Wenhui Fan、Natalia Vélez、Qingyun Wu、Huazheng Wang、Thomas L Griffiths、Mengdi Wang。2024年。具身大语言模型智能体在组织化的团队中学习合作。ArXiv预印本，abs/2403.12482。

丹·亨德里克斯，科林·伯恩斯，史蒂文·巴萨特，安迪·邹，曼塔斯·马泽伊卡，宋丹，以及雅各布·斯坦哈特。2021年。衡量大规模多任务语言理解。收录于第9届国际学习表示会议，ICLR 2021，虚拟会议，奥地利，2021年5月3日至7日。OpenReview.net。

洪思睿, 朱格明辰, 乔纳森·陈, 郑夏武, 程宇恒, 王金陵, 张策瑶, 王紫莉, 丘成桐, 林白舰, 李洋

Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. [MetaGPT: Meta programming for a multi-agent collaborative framework](#). In *The Twelfth International Conference on Learning Representations*.

Shengchao Hu, Li Shen, Ya Zhang, and Dacheng Tao. 2024. [Learning multi-agent communication from graph modeling perspective](#). In *The Twelfth International Conference on Learning Representations*.

Yue Hu, Yuzhu Cai, Yaxin Du, Xinyu Zhu, Xiangrui Liu, Zijie Yu, Yuchen Hou, Shuo Tang, and Siheng Chen. 2025. [Self-evolving multi-agent networks for software development](#). In *The Thirteenth International Conference on Learning Representations*.

Yoichi Ishibashi and Yoshimasa Nishimura. 2024. [Self-organized agents: A llm multi-agent framework toward ultra large-scale code generation and optimization](#). *ArXiv preprint*, abs/2404.02183.

Dongfu Jiang, Xiang Ren, and Bill Yuchen Lin. 2023. [LLM-blender: Ensembling large language models with pairwise ranking and generative fusion](#). In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14165–14178, Toronto, Canada. Association for Computational Linguistics.

junyou li, Qin Zhang, Yangbin Yu, QIANG FU, and Deheng Ye. 2024. [More agents is all you need](#). *Transactions on Machine Learning Research*.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan A, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. [DSPy: Compiling declarative language model calls into state-of-the-art pipelines](#). In *The Twelfth International Conference on Learning Representations*.

Steve WJ Kozlowski and Daniel R Ilgen. 2006. [Enhancing the effectiveness of work groups and teams](#). *Psychological science in the public interest*, 7(3):77–124.

Yunxuan Li, Yibing Du, Jiageng Zhang, Le Hou, Peter Grabowski, Yeqing Li, and Eugene Ie. 2024. [Improving multi-agent debate with sparse communication topology](#). In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 7281–7294, Miami, Florida, USA. Association for Computational Linguistics.

Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Shuming Shi, and Zhaopeng Tu. 2024. [Encouraging divergent thinking in large language models through multi-agent debate](#). In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 17889–17904, Miami, Florida, USA. Association for Computational Linguistics.

Wang Ling, Dani Yogatama, Chris Dyer, and Phil Blunsom. 2017. [Program induction by rationale generation: Learning to solve and explain algebraic word problems](#). In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 158–167, Vancouver, Canada. Association for Computational Linguistics.

Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024a. [Deepseek-v3 technical report](#). *ArXiv preprint*, abs/2412.19437.

Yuntao Liu, Yong Dou, Yuan Li, Xinhai Xu, and Donghong Liu. 2022. [Temporal dynamic weighted graph convolution for multi-agent reinforcement learning](#). In *Proceedings of the Annual Meeting of the Cognitive Science Society*, volume 44.

Zijun Liu, Yanzhe Zhang, Peng Li, Yang Liu, and Diyi Yang. 2024b. [A dynamic LLM-powered agent network for task-oriented agent collaboration](#). In *First Conference on Language Modeling*.

E. Locke. 2011. *Handbook of Principles of Organizational Behavior: Indispensable Knowledge for Evidence-Based Management*. Wiley.

Zhao Mandi, Shreeya Jain, and Shuran Song. 2024. [Roco: Dialectic multi-robot collaboration with large language models](#). In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 286–299.

Arkil Patel, Satwik Bhattamishra, and Navin Goyal. 2021. [Are NLP models really able to solve simple math word problems?](#) In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 2080–2094, Online. Association for Computational Linguistics.

Emanuele Pesce and Giovanni Montana. 2023. [Learning multi-agent coordination through connectivity-driven communication](#). *Machine Learning*, 112(2):483–514.

Chen Qian, Yufan Dang, Jiahao Li, Wei Liu, Zihao Xie, YiFei Wang, Weize Chen, Cheng Yang, Xin Cong, Xiaoyin Che, Zhiyuan Liu, and Maosong Sun. 2024a. [Experiential co-learning of software-developing agents](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 5628–5640, Bangkok, Thailand. Association for Computational Linguistics.

Chen Qian, Jiahao Li, Yufan Dang, Wei Liu, YiFei Wang, Zihao Xie, Weize Chen, Cheng Yang, Yingli Zhang, Zhiyuan Liu, et al. 2024b. [Iterative experience refinement of software-developing agents](#). *ArXiv preprint*, abs/2405.04219.

Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng

周, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, 和 于尔根·施密德胡伯. 2024年. [MetaGPT: Meta pro- gramming for a 多智能体协作框架](#). In 第十二届学习表征国际会议.

Shengchao Hu, Li Shen, Ya Zhang, 和 Dacheng Tao. 2024年. [从图建模视角学习多智能体通信](#). 载于第十二届学习表征国际会议.

Yue Hu, Yuzhu Cai, Yaxin Du, Xinyu Zhu, Xiangrui Liu, Zijie Yu, Yuchen Hou, Shuo Tang, 和 Siheng Chen. 2025年. [用于软件开发自进化多智能体网络](#). 载于第十三届学习表征国际会议.

Yoichi Ishibashi 和 Yoshimasa Nishimura. 2024年. [自组织智能体：一个面向超大规模代码生成与优化大语言模型多智能体框架](#). ArXiv预印本, abs/2404.02183.

Dongfu Jiang, Xiang Ren, 和 Bill Yuchen Lin. 2023年. [LLM-blender: 通过成对排名和生成式融合集成大型语言模型](#). 载于第61届计算语言学协会年会论文集（第一卷：长论文），第14165–14178页，加拿大多伦多. 计算语言学协会.

junyou li, Qin Zhang, Yangbin Yu, QIANG FU, 和 Deheng Ye. 2024年. [更多的智能体是你所需的一切](#). Transactions on Machine Learning Research.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan A, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, 和 Christopher Potts. 2024年. [DSPy: 将声明式语言模型调用编译为最先进的流水线](#). In 第十二届学习表征国际会议.

Steve WJ 科兹洛夫斯基 和 Daniel R 伊尔根. 2006年. [增强工作群体和团队的有效性](#). Psychological science inthe public interest, 7(3):77– 124.

Yunxuan Li, Yibing Du, Jiageng Zhang, Le Hou, Peter Grabowski, Yeqing Li, 和 Eugene Ie. 2024年. [改进多智能体辩论的稀疏通信拓扑](#). In Findings of the Association for Com- putational Linguistics: EMNLP 2024, pages 7281– 7294, 迈阿密, 佛罗里达, 美国. Association for Compu- tational Linguistics.

Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Shu ming Shi, 以及 Zhaopeng Tu. 2024年. [通过多智能体辩论鼓励大型语言模型的发散性思维](#). 载于《2024年自然语言处理实证方法会议论文集》，第17889–17904页，美国佛罗里达迈阿密. 计算语言学协会.

王林, 达尼·约加塔玛, 克里斯·戴尔, 以及 Phil Blunsom. 2017年. 通过推理生成进行程序归纳：学习与解释代数文字题. 载于《计算语言学协会第55届年会论文集（第1卷：长论文）》，第158–167页，加拿大温哥华. 计算语言学协会。

刘艾鑫, 冯北, 薛兵, 王炳轩, 吴博超, ChengdaLu, 赵成刚, 邓承奇, 张晨宇, 阮冲, et al. 2024a. [Deepseek-v3 技术报告](#). ArXiv预印本, abs/2412.19437.

刘云涛, 窦勇, 李源, 许新海, 和刘东红. 2022. [时变动态加权图卷积用于多智能体强化学习](#). 载于《认知科学学会年会论文集》，卷44.

刘志军, 张岩哲, 李鹏, 刘洋, 和杨一迪. 2024b. [用于面向任务的智能体协作的动态大语言模型驱动智能体网络](#). 载于首届语言建模会议.

E. 洛克. 2011. 《组织行为原则手册：循证管理必备知识》. 威利出版社.

Zhao Mandi, 希里娅·贾恩, 和宋舒然. 2024. [Roco：基于大语言模型的辩证多机器人协作](#). 载于2024年IEEE机器人与自动化国际会议 (ICRA), 页码286–299.

阿尔基尔·帕特尔, 萨特维·巴塔米什拉, 和纳文·戈亚尔. 2021年. [NLP模型真的能解决简单数学文字题吗？](#) In Proceedings of the 2021 Conference of the 北美计算语言学协会分会: 人类语言技术, 第2080–2094页，在线. 计算语言学协会。

埃马努埃莱·佩斯切 和 乔瓦尼·蒙塔纳. 2023年. [通过连通性驱动的通信学习多智能体协调](#). 机器学习, 112(2):483–514.

钱晨, 党宇凡, 李家豪, 刘伟, 谢子豪, 王一飞, 陈伟泽, 杨成, 丛信, 车晓音, 刘知远, 和孙茂松. 2024a. [软件开发智能体的经验共学习](#). In Proceedings of the 第62届计算语言学协会年会 (第一卷：长论文), 第5628–5640页，泰国曼谷. 计算语言学协会。

钱晨, 李家豪, 党宇凡, 刘伟, 王一飞, 谢子豪,陈伟泽, 杨成, 张颖丽, 刘知远, 等 . 2024b. [软件开发智能体的迭代经验精炼](#). ArXiv预印本, abs/2405.04219.

钱晨, 刘伟, 刘宏章, 陈诺, 党宇凡, 李家豪, 杨成, 陈伟泽, 玉生

翻译服务由 ChatECNU 大语言模型平台提供 <https://chat.ecnu.edu.cn/html/>
技术方案基于 PDFMathTranslate-next 项目 <https://github.com/PDFMathTranslate/PDFMathTranslate-next>

Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024c. [ChatDev: Communicative agents for software development](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15174–15186, Bangkok, Thailand. Association for Computational Linguistics.

Chen Qian, Zihao Xie, YiFei Wang, Wei Liu, Kunlun Zhu, Hanchen Xia, Yufan Dang, Zhuoyun Du, Weize Chen, Cheng Yang, Zhiyuan Liu, and Maosong Sun. 2025. [Scaling large language model-based multi-agent collaboration](#). In *The Thirteenth International Conference on Learning Representations*.

Subhro Roy and Dan Roth. 2015. [Solving general arithmetic word problems](#). In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pages 1743–1752, Lisbon, Portugal. Association for Computational Linguistics.

Gemini Team, Petko Georgiev, Ving Ian Lei, Ryan Burnell, Libin Bai, Anmol Gulati, Garrett Tanzer, Damien Vincent, Zhufeng Pan, Shibo Wang, et al. 2024. [Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context](#). *ArXiv preprint*, abs/2403.05530.

Junlin Wang, Jue Wang, Ben Athiwaratkun, Ce Zhang, and James Zou. 2025a. [Mixture-of-agents enhances large language model capabilities](#). In *The Thirteenth International Conference on Learning Representations*.

Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. 2024a. [A survey on large language model based autonomous agents](#). *Frontiers of Computer Science*, 18(6):186345.

Yutong Wang, Jiali Zeng, Xuebo Liu, Fandong Meng, Jie Zhou, and Min Zhang. 2024b. [TasTe: Teaching large language models to translate through self-reflection](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6144–6158, Bangkok, Thailand. Association for Computational Linguistics.

Yutong Wang, Jiali Zeng, Xuebo Liu, Derek F. Wong, Fandong Meng, Jie Zhou, and Min Zhang. 2025b. [DeITA: An online document-level translation agent based on multi-level memory](#). In *The Thirteenth International Conference on Learning Representations*.

Zhenhailong Wang, Shaoguang Mao, Wenshan Wu, Tao Ge, Furu Wei, and Heng Ji. 2024c. [Unleashing the emergent cognitive synergy in large language models: A task-solving agent through multi-persona self-collaboration](#). In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 257–279, Mexico City, Mexico. Association for Computational Linguistics.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. [Chain-of-thought prompting elicits reasoning in large language models](#). In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. 2023. [Auto-gen: Enabling next-gen llm applications via multi-agent conversation framework](#). *ArXiv preprint*, abs/2308.08155.

An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. 2024. [Qwen2.5 technical report](#). *ArXiv preprint*, abs/2412.15115.

Haoran You, Chaojian Li, Pengfei Xu, Yonggan Fu, Yue Wang, Xiaohan Chen, Richard G. Baraniuk, Zhangyang Wang, and Yingyan Lin. 2020. [Drawing early-bird tickets: Toward more efficient training of deep networks](#). In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net.

Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. 2023. [Self-taught optimizer \(stop\): Recursively self-improving code generation](#). *ArXiv preprint*, abs/2310.02304.

Guibin Zhang, Yanwei Yue, Zhixun Li, Sukwon Yun, Guancheng Wan, Kun Wang, Dawei Cheng, Jeffrey Xu Yu, and Tianlong Chen. 2025. [Cut the crap: An economical communication pipeline for LLM-based multi-agent systems](#). In *The Thirteenth International Conference on Learning Representations*.

Guibin Zhang, Yanwei Yue, Kun Wang, Junfeng Fang, Yongduo Sui, Kai Wang, Yuxuan Liang, Dawei Cheng, Shirui Pan, and Tianlong Chen. 2024a. [Two heads are better than one: Boosting graph sparse training via semantic and topological awareness](#). In *Forty-first International Conference on Machine Learning*.

Hongxin Zhang, Weihua Du, Jiaming Shan, Qinzhong Zhou, Yilun Du, Joshua B. Tenenbaum, Tianmin Shu, and Chuang Gan. 2024b. [Building cooperative embodied agents modularly with large language models](#). In *The Twelfth International Conference on Learning Representations*.

Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen

苏, 丛信, 许巨元, 李大辉, 刘知远, 孙茂松. 2024年c. ChatDev: 用于软件开发的通信智能体. 载于《计算语言学协会第62届年会论文集》(第一卷: 长论文), 第15174–15186页, 泰国曼谷. 计算语言学协会.

钱晨, 谢子豪, 王一飞, 刘伟, 朱昆仑, 夏汉辰, 党宇凡, 杜卓云, 陈伟泽, 杨成, 刘知远, and 孙茂松. 2025年. 基于大规模语言模型的多智能体协作扩展. 收录于《第十三届学习表征国际会议》。

Subhro Roy and Dan Roth. 2015. 求解一般算术文字问题. 收录于《2015年自然语言处理实证方法会议论文集》, 第1743–1752页, 葡萄牙里斯本. 计算语言学协会.

Gemini Team, Petko Georgiev, Ving Ian Lei, Ryan Burnell, Libin Bai, Anmol Gulati, Garrett Tanzer, Damien Vincent, Zhufeng Pan, Shibo Wang, et al. 2024年. Gemini 1.5: 解锁数百万词元上下文中的多模态理解. ArXiv预印本, abs/2403.05530.

Junlin Wang, Jue Wang, Ben Athiwaratkun, Ce Zhang, and James Zou. 2025a. 智能体混合增强大语言模型能力. In The Thirteenth International Conference on Learning Representations.

Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. 2024a. 基于大语言模型的自主智能体综述. 计算机科学前沿, 18(6):186345.

王雨桐, Jiali Zeng, Xuebo Liu, Fandong Meng, 周, and 张敏. 2024b. TasTe: Teaching large language models to translate through self-reflection. In Proceedings of the 62nd Annual Meeting of the 计算语言学协会 (Volume 1: Long Papers), pages 6144–6158, 泰国曼谷. 计算语言学协会.

Yutong Wang, Jiali Zeng, Xuebo Liu, Derek F. Wong, Fandong Meng, Jie Zhou, and 张敏. 2025年b. DeITA: 基于多层记忆的在线文档级翻译智能体. In The Thirteenth International Conference on Learning Representations.

Zhenhailong Wang, Shaoguang Mao, Wenshan Wu, Tao Ge, Furu Wei, and Heng Ji. 2024c. Unleashing the emergent cognitive synergy in large language models: A task-solving agent through multi-persona self-collaboration. 载于 Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Tech- nologies (Volume 1: Long Papers), 第 257–279 页, 墨西哥墨西哥城. 计算语言学协会.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. 载于 Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, 神经信息处理系统大会 2022, 美国路易斯安那州新奥尔良, 2022年11月28日 - 12月9日.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. 2023年. Auto-gen: Enabling next-gen 大语言模型应用 via multi-agent conversation framework. ArXiv预印本, abs/2308.08155.

An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jingren 周, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. 2024年. Qwen2.5 技术报告. ArXiv预印本, abs/2412.15115.

Haoran You, Chaojian Li, Pengfei Xu, Yonggan Fu, Yue Wang, Xiaohan Chen, Richard G. Baraniuk, Zhangyang Wang, and Yingyan Lin. 2020年. Drawing early-bird tickets: Toward more efficient training of deep networks. In 第八届学习表征国际会议, 国际学习表征会议 2020, 埃塞俄比亚的斯亚贝巴, 2020年4月26-30日. OpenReview.net.

Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. 2023年. 自教学优化器 (STOP): 递归自我改进 代码生成. ArXiv预印本, abs/2310.02304.

Guibin Zhang, Yanwei Yue, Zhixun Li, Sukwon Yun, Guancheng Wan, Kun Wang, Dawei Cheng, Jeffrey Xu Yu, and Tianlong Chen. 2025年. Cut the crap: An economical communication pipeline for 大语言模型- 基于多智能体系统. In The Thirteenth International Conference on Learning Representations.

Guibin Zhang, Yanwei Yue, Kun Wang, 房俊峰, 岁永多, 王凯, 梁宇轩, Dawei Cheng, 潘仕瑞, and Tianlong Chen. 2024a. Two heads are better than one: Boosting graph sparse training via semantic and topological awareness. In 第四十一次国际机器学习大会.

张红新, 杜伟华, 单嘉铭, 周钦鸿, 杜一伦, 乔舒亚·B·特宁鲍姆, 书天民, and 甘闯. 2024b. Building cooperative embodied agents modularly with large language models. In 第十二届学习表征国际会议.

朱格明辰, 王文一, 路易·基施, 弗朗切斯科·法乔, 德米特里·基兹布林, 和于尔根

Schmidhuber. 2024. [GPTSwarm: Language agents as optimizable graphs](#). In *Forty-first International Conference on Machine Learning*.

A Appendix

A.1 Optimization Algorithm of AgentDropout

Algorithm 1 presents the detailed pseudo code of AgentDropout. The input parameters for this algorithm mainly include communication graph $\mathcal{G}$ to be optimized, along with its intra- and inter-round adjacency matrices $\hat{\mathcal{A}}_{\text{intra}}, \hat{\mathcal{A}}_{\text{inter}}$. The output is the optimized communication graph $\hat{\mathcal{G}}$.

During NodeDropout, we first call the `Initialize()` function to initialize the adjacency matrices by setting the relevant elements to 0.5 (Line 1). We then train the intra-round matrices to optimize the task performance of the communication graphs sampled from these matrices (Lines 2 ~5). After the training process, we identify the nodes in each communication round with the smallest in- and out-degree (Line 6) and remove them, along with all the edges connected to them, by invoking the `NodeDropout()` function (Line 7).

During EdgeDropout, we initialize the intra- and inter-round adjacency matrices again (Line 8) and train both of them to optimize task performance and graph sparsity simultaneously (Lines 9 ~12). Next, we identify the intra- and inter-round edges with the smallest weights in their respective matrices (Line 13) and remove them from the graph by calling the `EdgeDropout()` function (Line 14). Finally, we sample the communication graph used in actual evaluation by calling the `DAGSample()` function, which samples the graph by treating the weights in the matrices as sampling possibility and guarantees the output to be a DAG graph (Line 15).

A.2 Case Study

Given a multiple-choice physics question from the MMLU dataset, in the first round of interaction, the Knowledgeable Expert provides an initial analysis, emphasizing the need to search for information related to the equation and relevant atomic properties. The Critic critiques the response of the Knowledgeable Expert, pointing out the lack of practical analysis and application, and suggests further refinement of the reasoning. Upon receiving feedback, the Mathematician conducts a detailed analysis of the problem, noting that the equation cannot be applied to H^- , and discusses variations

across different atoms, providing more specific reasoning. The Psychologist offers feedback from a psychological perspective, affirming the process of critique and revision, and encourages more comprehensive critical thinking. In the second round of interaction, the Historian briefly summarizes the reasons. The Critic revisits the perspectives of other agents, suggesting that the approach of Mathematicians to different atoms could be further improved, and emphasizes the impact of atomic properties on the equation. After receiving feedback, the Mathematician revises the analysis, clarifies which atoms can be addressed by modifying the application of the Rydberg equation, and concludes that the Hydride ion (H^-) is the only unsuitable option. The Historian in the first round and the Knowledgeable Expert in the second round are dropped out.

A.3 Token Efficiency

As shown in Table 7, AgentDropout significantly reduces token consumption compared to the multi-round method $\text{MAS}_{\text{round}=T}$. Unlike AgentPrune, which reduces only prompt tokens, AgentDropout reduces both prompt and completion tokens. In the Llama3 model, it achieves a 21.4% reduction in prompt tokens and 16.1% in completion tokens. For Qwen2.5 and Deepseek-V3, the reductions are 24.4% and 18.9% in prompt tokens, and 21.4% and 17.6% in completion tokens, respectively, demonstrating its token efficiency.

A.4 Different Communication Topologies

In the main experiments, we initialize the intra-round communication graph using a fully connected graph. In the analysis experiments, we supplement this by conducting additional experiments with intra-round graphs initialized using layered graphs and random graphs, as shown in Figure 3. For both fully connected graphs and random graphs, we first process the communication topology into a DAG using `DAGSample`, and then perform inference and communication based on the topological order (Zhuge et al., 2024; Qian et al., 2025). For layered graphs, similar to multi-layer perceptrons, the agents in the first layer perform inference first, followed by message passing to the agents in the second layer, and so on.

施密德胡伯. 2024年. GPTSwarm: 语言智能体作为可优化图. 收录于第四十一次国际机器学习大会.

A 附录

A.1 AgentDropout 的优化算法

算法1 给出了 AgentDropout 的详细伪代码。该算法的输入参数主要包括待优化的通信图 $\mathcal{G}$, 以及其轮内和轮间邻接矩阵 $\mathcal{A}_{\text{intra}}, \mathcal{A}_{\text{inter}}$ 。输出为优化后的通信图 $\hat{\mathcal{G}}$ 。

在 NodeDropout 期间, 我们首先调用 `Initialize()` 函数将相关元素设置为 0.5, 从而初始化邻接矩阵 (第 1 行)。然后, 我们训练轮内矩阵, 以优化从这些矩阵中采样的通信图的任务性能 (第2 ~5行)。训练过程完成后, 我们识别每个通信轮次中入度和出度最小的节点 (第 6 行), 并通过调用 `NodeDropout()` 函数 (第 7 行) 移除这些节点及其连接的所有边。

在 EdgeDropout 期间, 我们再次初始化轮内和轮间邻接矩阵 (第 8 行), 并同时训练这两个矩阵以优化任务性能和图稀疏度 (第 9 ~12 行)。接下来, 我们识别各自矩阵中权重最小的轮内和轮间边 (第 13 行), 并通过调用 `EdgeDropout()` 函数 (第 14 行) 将它们从图中移除。最后, 我们调用 `DAGSample()` 函数来采样用于实际评估的通信图, 该函数将矩阵中的权重作为采样可能性进行采样, 并确保输出为一个有向无环图 (第 15 行)。

A.2 案例研究

针对来自 MMLU 数据集的一道多项选择物理题, 在首轮交互中, 知识渊博专家提供初步分析, 强调需要查找与该方程及相关原子性质有关的信息。评论家对知识渊博专家的回复进行批评, 指出其缺乏实际分析和应用, 并建议进一步完善推理。在收到反馈后, 数学家对该问题进行详细分析, 指出该方程不能应用于 H^- , 并讨论其在不同原子间的差异

, 提供更具针对性的推理。心理学家从心理学视角给出反馈, 肯定批评与修改的过程, 并鼓励进行更全面的批判性思维。在第二轮交互中, 历史学家简要总结原因。评论家重新审视其他智能体的观点, 建议进一步改进数学家对不同原子的处理方法, 并强调原子性质对该方程的影响。在收到反馈后, 数学家修订了分析, 澄清了通过修改里德伯方程的应用可以处理哪些原子, 并得出结论, 氢化物离子 (H^-) 是唯一不合适的选项。第一轮の数学家和第二轮的知识渊博专家被移除。

A.3 令牌效率

如表 7 所示, 与多轮次方法 $\text{MAS}_{\text{round}=T}$ 相比, AgentDropout 显著降低了令牌消耗。与仅减少提示令牌的 AgentPrune 不同, AgentDropout 同时减少了提示令牌和补全令牌。在 Llama3 模型中, 提示令牌减少了 21.4%, 补全令牌减少了 16.1%。对于 Qwen2.5 和 Deepseek-V3, 提示令牌分别减少了 24.4% 和 18.9%, 补全令牌分别减少了 21.4% 和 17.6%, 证明了其令牌效率。

A.4 不同通信拓扑

在主要实验中, 我们使用全连接图初始化轮内通信图。在分析实验中, 我们补充进行了额外实验, 使用分层图和随机图初始化轮内图, 如图 3 所示。对于全连接图和随机图, 我们首先使用 `DAGSample` 将通信拓扑处理为有向无环图, 然后基于拓扑顺序进行推理和通信 (Zhuge et al., 2024年; Qian et al., 2025年)。对于分层图, 类似于多层感知器, 第一层的智能体先执行推理, 随后向第二层的智能体进行消息传递, 依此类推。

Algorithm 1: Communication Graph Dropout

Input: Initial communication graph $\mathcal{G} = (\bigcup_t V^{(t)}, (\bigcup_t E_{\text{intra}}^{(t)} \cup (\bigcup_t E_{\text{inter}}^{(t)}), \bigcup_t F^{(t)})$, Initial adjacency matrices $\tilde{\mathcal{A}}_{\text{intra}} = \{\tilde{\mathcal{A}}_{\text{intra}}^{(t)}\}_t$, $\tilde{\mathcal{A}}_{\text{inter}} = \{\tilde{\mathcal{A}}_{\text{inter}}^{(t)}\}_t$, Training steps K , Sampling times M , Learning rate η , Node dropout rate α , Edge dropout rate β

Output: Communication graph after dropout $\hat{\mathcal{G}}$

```

// Node Dropout
1 Initialize  $(\tilde{\mathcal{A}}_{\text{intra}}, \tilde{\mathcal{A}}_{\text{inter}})$  // Set the elements to 0.5
2 for iter  $k \leftarrow 1$  to  $K$  do
3    $\{\mathcal{G}_m\}_{m=1}^M \leftarrow \{\text{DAGSample}(\tilde{\mathcal{A}}_{\text{intra}}, \tilde{\mathcal{A}}_{\text{inter}})\}_{m=1}^M$  // Sample  $M$  communication graphs
4    $\tilde{\mathcal{A}}_{\text{intra}} \leftarrow \tilde{\mathcal{A}}_{\text{intra}} + \eta \cdot \frac{1}{M} \sum_{m=1}^M \mu(\mathcal{G}_m) \nabla_{\tilde{\mathcal{A}}_{\text{intra}}} \log(p_{\tilde{\mathcal{A}}_{\text{intra}}}(\mathcal{G}_m))$  // Optimize  $\tilde{\mathcal{A}}_{\text{intra}}$  according to Equation (1)
5 end
6  $\mathcal{V}_{\Delta} = \bigcup_t (V^{(t)} \setminus \text{TopkNodes}(\tilde{\mathcal{A}}_{\text{intra}}^{(t)}, 1 - \alpha))$  // Select nodes with small degrees
7  $\tilde{\mathcal{A}}_{\text{intra}}, \tilde{\mathcal{A}}_{\text{inter}} \leftarrow \text{NodeDropout}(\tilde{\mathcal{A}}_{\text{intra}}, \tilde{\mathcal{A}}_{\text{inter}}, \mathcal{V}_{\Delta})$  // Remove nodes from the graph
// Edge Dropout
8 Initialize  $(\tilde{\mathcal{A}}_{\text{intra}}, \tilde{\mathcal{A}}_{\text{inter}})$ 
9 for iter  $k \leftarrow 1$  to  $K$  do
10   $\{\mathcal{G}_m\}_{m=1}^M \leftarrow \{\text{DAGSample}(\tilde{\mathcal{A}}_{\text{intra}}, \tilde{\mathcal{A}}_{\text{inter}})\}_{m=1}^M$  // Sample  $M$  communication graphs
11   $\{\tilde{\mathcal{A}}_{\text{intra}}, \tilde{\mathcal{A}}_{\text{inter}}\} \leftarrow \{\tilde{\mathcal{A}}_{\text{intra}}, \tilde{\mathcal{A}}_{\text{inter}}\} + \eta \cdot \frac{1}{M} \sum_{m=1}^M \mu(\mathcal{G}_m) \nabla_{\tilde{\mathcal{A}}_{\text{intra}}, \tilde{\mathcal{A}}_{\text{inter}}} \log(p_{\tilde{\mathcal{A}}_{\text{intra}}, \tilde{\mathcal{A}}_{\text{inter}}}(\mathcal{G}_m))$  // Optimize  $\tilde{\mathcal{A}}$  according to Equation (5)
12 end
13  $\mathcal{E}_{\Delta} = \bigcup_t (E_{\text{intra}}^{(t)} \setminus \text{TopkEdges}(\tilde{\mathcal{A}}_{\text{intra}}^{(t)}, 1 - \beta)) \cup \bigcup_t (E_{\text{inter}}^{(t)} \setminus \text{TopkEdges}(\tilde{\mathcal{A}}_{\text{inter}}^{(t)}, 1 - \beta))$  // Select edges with small weights
14  $\tilde{\mathcal{A}}_{\text{intra}}, \tilde{\mathcal{A}}_{\text{inter}} \leftarrow \text{EdgeDropout}(\tilde{\mathcal{A}}_{\text{intra}}, \tilde{\mathcal{A}}_{\text{inter}}, \mathcal{E}_{\Delta})$  // Remove edges from the graph
15  $\hat{\mathcal{G}} \leftarrow \text{DAGSample}(\tilde{\mathcal{A}}_{\text{intra}}, \tilde{\mathcal{A}}_{\text{inter}})$  // Sample the final communication DAG
16 return  $\hat{\mathcal{G}}$ 

```

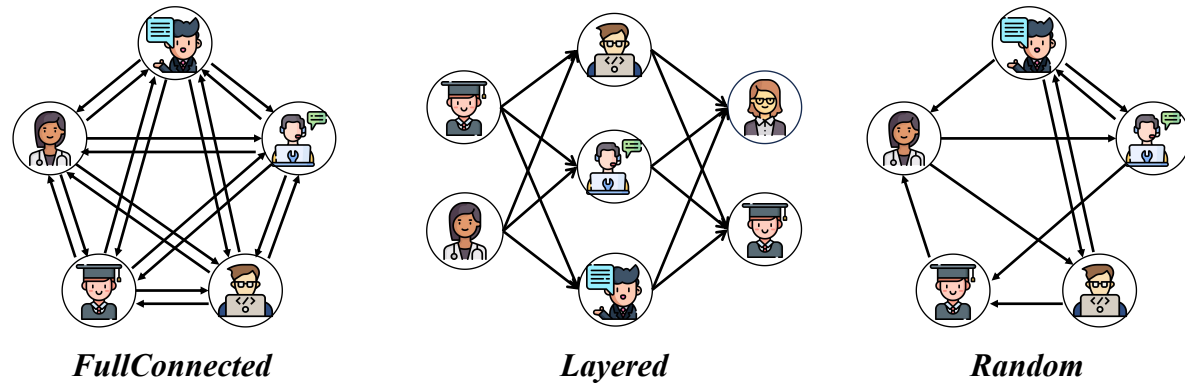


Figure 3: Full connected, layer and random graph as demonstrations of intra-round communication topologies.

算法1: 通信图Dropout 输入: 初始通信图 $\mathcal{G} = (\bigcup_t V^{(t)}, (\bigcup_t E_{\text{intra}}^{(t)} \cup (\bigcup_t E_{\text{inter}}^{(t)}), \bigcup_t F^{(t)})$, 初始邻接矩阵 $\tilde{\mathcal{A}}_{\text{intra}} = \{\tilde{\mathcal{A}}_{\text{intra}}^{(t)}\}_t$, $\tilde{\mathcal{A}}_{\text{inter}} = \{\tilde{\mathcal{A}}_{\text{inter}}^{(t)}\}_t$, 训练步数 K , 采样次数 M , 学习率 η , 节点 Dropout 率 α , 边 Dropout 率 β 输出: Dropout 后的通信图 $\hat{\mathcal{G}}$

```

// Node Dropout
1 Initialize  $(\tilde{\mathcal{A}}_{\text{intra}}, \tilde{\mathcal{A}}_{\text{inter}})$  // Set the elements to 0.52 for iter  $k \leftarrow 1$ 
to  $K$  do
   $\{\mathcal{G}_m\}_{m=1}^M \leftarrow \{\text{DAGSample}(\tilde{\mathcal{A}}_{\text{intra}}, \tilde{\mathcal{A}}_{\text{inter}})\}_{m=1}^M$  // Sample  $M$  communication graphs
   $\tilde{\mathcal{A}}_{\text{intra}} \leftarrow \tilde{\mathcal{A}}_{\text{intra}} + \eta \cdot \frac{1}{M} \sum_{m=1}^M \mu(\mathcal{G}_m) \nabla_{\tilde{\mathcal{A}}_{\text{intra}}} \log(p_{\tilde{\mathcal{A}}_{\text{intra}}}(\mathcal{G}_m))$  // Optimize  $\tilde{\mathcal{A}}_{\text{intra}}$  according to Equation (1)
5 end
6  $\mathcal{V}_{\Delta} = \bigcup_t (V^{(t)} \setminus \text{TopkNodes}(\tilde{\mathcal{A}}_{\text{intra}}^{(t)}, 1 - \alpha))$  // Select nodes with small degrees
7  $\tilde{\mathcal{A}}_{\text{intra}}, \tilde{\mathcal{A}}_{\text{inter}} \leftarrow \text{NodeDropout}(\tilde{\mathcal{A}}_{\text{intra}}, \tilde{\mathcal{A}}_{\text{inter}}, \mathcal{V}_{\Delta})$  // Remove nodes from the graph
// Edge Dropout
8 Initialize  $(\tilde{\mathcal{A}}_{\text{intra}}, \tilde{\mathcal{A}}_{\text{inter}})$ 
9 for iter  $k \leftarrow 1$  to  $K$  do
10   $\{\mathcal{G}_m\}_{m=1}^M \leftarrow \{\text{DAGSample}(\tilde{\mathcal{A}}_{\text{intra}}, \tilde{\mathcal{A}}_{\text{inter}})\}_{m=1}^M$  // Sample  $M$  communication graphs
11   $\{\tilde{\mathcal{A}}_{\text{intra}}, \tilde{\mathcal{A}}_{\text{inter}}\} \leftarrow \{\tilde{\mathcal{A}}_{\text{intra}}, \tilde{\mathcal{A}}_{\text{inter}}\} + \eta \cdot \frac{1}{M} \sum_{m=1}^M \mu(\mathcal{G}_m) \nabla_{\tilde{\mathcal{A}}_{\text{intra}}, \tilde{\mathcal{A}}_{\text{inter}}} \log(p_{\tilde{\mathcal{A}}_{\text{intra}}, \tilde{\mathcal{A}}_{\text{inter}}}(\mathcal{G}_m))$  // Optimize  $\tilde{\mathcal{A}}$  according to Equation (5)
12 end
13  $\mathcal{E}_{\Delta} = \bigcup_t (E_{\text{intra}}^{(t)} \setminus \text{TopkEdges}(\tilde{\mathcal{A}}_{\text{intra}}^{(t)}, 1 - \beta)) \cup \bigcup_t (E_{\text{inter}}^{(t)} \setminus \text{TopkEdges}(\tilde{\mathcal{A}}_{\text{inter}}^{(t)}, 1 - \beta))$  // Select edges with small weights
14  $\tilde{\mathcal{A}}_{\text{intra}}, \tilde{\mathcal{A}}_{\text{inter}} \leftarrow \text{EdgeDropout}(\tilde{\mathcal{A}}_{\text{intra}}, \tilde{\mathcal{A}}_{\text{inter}}, \mathcal{E}_{\Delta})$  // Remove edges from the graph
15  $\hat{\mathcal{G}} \leftarrow \text{DAGSample}(\tilde{\mathcal{A}}_{\text{intra}}, \tilde{\mathcal{A}}_{\text{inter}})$  // Sample the final communication DAG
16 return  $\hat{\mathcal{G}}$ 

```

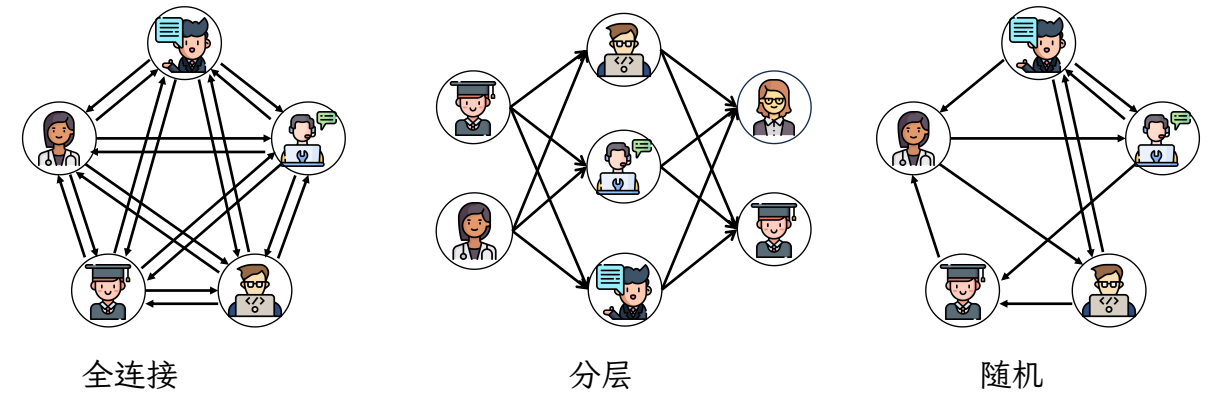


图 3: 全连接、分层和随机图，作为轮内通信拓扑的示例。

Method	MMLU		GSM8K		AQuA		MultiArith		SVAMP		HumanEval		Avg.	
	Ptok.	Ctok.	Ptok.	Ctok.	Ptok.	Ctok.	Ptok.	Ctok.	Ptok.	Ctok.	Ptok.	Ctok.	Ptok.	Ctok.
Base model: Llama3-8B-Instruct														
Vanilla	99K	44K	142K	209K	32K	89K	16K	15k	27K	19K	91K	25K	68K	67K
CoT	129K	70K	154K	337K	34K	105K	18K	23k	30K	50K	93K	29K	76K	102K
MAS _{round=1}	1.4M	355K	8.5M	1.9M	1.1M	390K	1.1M	218K	1.9M	402K	581K	134K	2.4M	567K
MAS _{round=T}	1.6M	387K	16M	3.4M	2.4M	745K	2.1M	388K	3.7M	721K	2.6M	492K	4.7M	1.0M
AgentPrune	1.3M	367K	15M	3.6M	2.0M	759K	1.9M	393K	3.4M	714K	1.5M	465K	4.2M	1.0M
AgentDropout	1.1M	333K	12M	2.8M	1.3M	634K	1.4M	312K	2.6M	594K	1.1M	359K	3.3M	839K
Base model: Qwen2.5-72B-Instruct														
Vanilla	99K	42K	142K	232K	32K	86K	16K	14k	27K	18K	91K	24K	68K	69K
CoT	129K	71K	154K	340K	34K	113K	18K	28k	30K	56K	93K	26K	76K	106K
MAS _{round=1}	966K	315K	9.0M	1.8M	1.4M	503K	1.1M	168K	1.8M	293K	271K	42K	2.4M	520K
MAS _{round=T}	2.2M	663K	18M	3.5M	2.9M	988K	2.0M	314K	3.4M	546K	814K	121K	4.9M	1.0M
AgentPrune	2.0M	665K	15M	3.3M	2.4M	995K	1.8M	313K	3.1M	517K	512K	83K	4.1M	979K
AgentDropout	1.3M	456K	11M	2.6M	2.0M	779K	1.6M	234K	2.4M	466K	421K	77K	3.1M	769K
Base model: Deepseek-V3-671B-Instruct														
Vanilla	99K	39K	137K	233K	30K	79K	16K	14K	26K	28K	92K	27K	67K	70K
CoT	129K	65K	148K	289K	34K	98K	17K	23K	29K	50K	93K	27K	75K	92K
MAS _{round=1}	845K	247K	7.9M	1.6M	1.2M	444K	951K	157K	1.6M	258K	339K	71K	2.1M	463K
MAS _{round=T}	2.1M	575K	16M	3.2M	2.7M	915K	1.9M	285K	3.0M	456K	1.3M	286K	4.5M	953K
AgentPrune	1.6M	582K	13M	3.2M	2.2M	902K	1.7M	296K	2.9M	498K	649K	206K	3.7M	947K
AgentDropout	1.0M	328K	11M	2.7M	1.6M	847K	1.4M	237K	2.3M	399K	469K	169K	3.0M	780K

Table 7: Token consumption comparison between AgentDropout and other baseline reasoning techniques. **Ptok.** denotes the number of prompt tokens for the agents, while **Ctok.** represents the number of completion tokens generated by the agents.

方法	MMLU		GSM8k		AQuA		MultiArith		SVAMP		HumanEval		Avg.	
	预训练	上下文	预训练	上下文	预训练	上下文	预训练	上下文	预训练	上下文	预训练	上下文	预训练	上下文
基础模型: Llama3-8B-Instruct														
原版	99K	44K	142K	209K	32K	89K	16K	15k	27K	19K	91K	25K	68K	67K
CoT	129K	70K	154K	337K	34K	105K	18K	23k	30K	50K	93K	29K	76K	102K
多智能体系统 _{round=1}	1.4M	355K	8.5M	1.9M	1.1M	390K	1.1M	218K	1.9M	402K	581K	134K	2.4M	567K
多智能体系统 _{round=T}	1.6M	387K	16M	3.4M	2.4M	745K	2.1M	388K	3.7M	721K	2.6M	492K	4.7M	1.0M
AgentPrune	1.3M	367K	15M	3.6M	2.0M	759K	1.9M	393K	3.4M	714K	1.5M	465K	4.2M	1.0M
AgentDropout	1.1M	333K	12M	2.8M	1.3M	634K	1.4M	312K	2.6M	594K	1.1M	359K	3.3M	839K
基础模型: Qwen2.5-72B-Instruct														
原版	99K	42K	142K	232K	32K	86K	16K	14k	27K	18K	91K	24K	68K	69K
CoT	129K	71K	154K	340K	34K	113K	18K	28k	30K	56K	93K	26K	76K	106K
多智能体系统 _{round=1}	966K	315K	9.0M	1.8M	1.4M	503K	1.1M	168K	1.8M	293K	271K	42K	2.4M	520K
多智能体系统 _{round=T}	2.2M	663K	18M	3.5M	2.9M	988K	2.0M	314K	3.4M	546K	814K	121K	4.9M	1.0M
AgentPrune	2.0M	665K	15M	3.3M	2.4M	995K	1.8M	313K	3.1M	517K	512K	83K	4.1M	979K
AgentDropout	1.3M	456K	11M	2.6M	2.0M	779K	1.6M	234K	2.4M	466K	421K	77K	3.1M	769K
基础模型: Deepseek-V3-671B-Instruct														
原版	99K	39K	137K	233K	30K	79K	16K	14K	26K	28K	92K	27K	67K	70K
CoT	129K	65K	148K	289K	34K	98K	17K	23K	29K	50K	93K	27K	75K	92K
多智能体系统 _{round=1}	845K	247K	7.9M	1.6M	1.2M	444K	951K	157K	1.6M	258K	339K	71K	2.1M	463K
多智能体系统 _{round=T}	2.1M	575K	16M	3.2M	2.7M	915K	1.9M	285K	3.0M	456K	1.3M	286K	4.5M	953K
AgentPrune	1.6M	582K	13M	3.2M	2.2M	902K	1.7M	296K	2.9M	498K	649K	206K	3.7M	947K
AgentDropout	1.0M	328K	11M	2.7M	1.6M	847K	1.4M	237K	2.3M	399K	469K	169K	3.0M	780K

表7: AgentDropout 与其他基线推理技术之间的令牌消耗对比。Ptok. 表示智能体的提示令牌数量, Ctok. 表示智能体生成的补全令牌数量。

Task (1/12) &  Knowledgeable Expert 3FeF: Round 1 (2/12)

The Rydberg equation $v = R_H(1/n_1^2 - 1/n_2^2)$ accurately predicts the UV-visible emission spectrum of the hydrogen atom. A form of the Rydberg equation may also be used to predict the UV-visible emission for all of the following EXCEPT

Option A: hydride ion, H-
Option B: deuterium atom, D
Option C: tritium atom, T
Option D: helium cation, He+

A fascinating topic!

To solve this problem, we need to search for entities related to the Rydberg equation and atomic properties. Here are the key entities to search in Wikipedia:

@Rydberg equation@
@Hydrogen atom@
@Hydride ion (H-)@
@Deuterium atom (D)@
@Tritium atom (T)@
@Helium cation (He+)@

These entities will provide us with the necessary information to determine which option(s) is/are incorrect.

任务 (1/12) &  知识渊博的专家 3FeF: 第1轮 (2/12)

里德伯方程 $v = R_H(1/n_1^2 - 1/n_2^2)$ 能够准确预测氢原子的紫外-可见发射光谱。里德伯方程的一种形式也可用于预测以下所有情形的紫外-可见发射, 除了

选项 A: 氢负离子, H- 选项 B: 氘原子, D 选项 C: 氚原子, T 选项 D: 氦离子, He+

一个引人入胜的话题!

为了解决这个问题, 我们需要搜索与里德伯方程和原子性质相关的实体。以下是应在维基百科中搜索的关键实体:

@里德伯方程@ @氢原子@ @氢负离子 (H-)@ @氘原子 (D)@ @氚原子 (T)@ @氦离子 (He+)@ 这些实体将为我们提供确定哪些选项是错误信息所需的信息。



Critic 6PWE: Round 1 (3/12)

As a critic, I'll analyze the output of Agent 3FeF and identify potential issues:

1. Missing analysis: The output doesn't provide any actual analysis or reasoning for why the Rydberg equation might not apply to the mentioned options. It merely states that searching for entities related to the Rydberg equation and atomic properties is necessary, without actually applying those concepts to the problem.
2. Lack of specific mention: The output doesn't mention a specific aspect of the Rydberg equation that might not be applicable to the mentioned options. It's unclear what part of the equation might be affected by the changes in atomic properties.
3. Overemphasis on searching: While searching for relevant Wikipedia articles is a good idea, it's not enough to solve the problem. The output could benefit from a more in-depth analysis of the Rydberg equation and its applications to different atoms.
4. Unclear role: Agent 3FeF claims to be a "Knowledgeable Expert" but doesn't demonstrate any expert-level knowledge or critical thinking in their output.

To improve the analysis, Agent 3FeF could provide a more comprehensive approach, such as:

1. Identifying specific aspects of the Rydberg equation that might not be applicable to the mentioned options (e.g., the assumption of a single electron or a specific type of atomic orbital)
2. Providing examples or analogies to illustrate why the Rydberg equation might not work for certain options
3. Offering alternative solutions or approaches to understanding the emission spectra of the mentioned atoms
4. Explicitly stating the limitations and assumptions involved in applying the Rydberg equation to different atoms

By addressing these issues, Agent 3FeF can enhance the quality of their analysis and provide a more robust solution to the problem.



批评者6PWE: 第1轮 (3/12)

作为一名评论家, 我将分析智能体3FeF的输出, 并识别潜在问题:

1. 缺少分析: 输出未提供任何关于里德伯方程为何不适用于所提及选项的实际分析或推理。它仅指出搜索与里德伯方程和原子性质相关的实体是必要的, 而未将这些概念应用于问题。
2. 缺乏具体说明: 输出未提及里德伯方程中可能与所列选项不相关的具体方面。不清楚方程的哪一部分可能受到原子性质变化的影响。
3. 对检索的过度强调: 虽然查找相关的维基百科文章是一个好主意, 但这不足以解决问题。输出内容可以通过对里德伯方程及其在不同原子中的应用进行更深入的分析而得到改善。
4. 角色不明确: 智能体3FeF自称是“知识渊博专家”, 但其输出中未展现出任何专家级知识或批判性思维。

为了改进分析, 智能体3FeF可以提供一种更全面的方法, 例如:

1. 识别里德伯方程中可能不适用于所述选项的特定方面 (例如, 单电子的假设或特定类型的原子轨道)
2. 提供示例或类比来说明里德伯方程为何可能不适用于某些选项
3. 提供替代方案或方法来理解所提及原子的发射光谱
4. 明确陈述将里德伯方程应用于不同原子时所涉及的局限性和假设

通过解决这些问题, 智能体3FeF 可以提升其分析质量, 并提供更稳健的问题解决方案。



Mathematician 32fy: Round 1 (4/12)

Thank you for the feedback from Agent 6PWE. As a mathematician, I must agree that my previous output was lacking in depth and analysis. Here's my revised answer:

The Rydberg equation, $v = R_H(1/n_1^2 - 1/n_2^2)$, accurately predicts the UV-visible emission spectrum of the hydrogen atom because it assumes a single electron in a hydrogen-like atom. However, when applying this equation to other atoms, such as hydride ion (H⁻), deuterium atom (D), tritium atom (T), and helium cation (He⁺), we must consider the following limitations:

1. The Rydberg equation assumes a single electron, which is not the case for the hydride ion (H⁻), which has two electrons. This means that the equation cannot accurately predict the emission spectrum of the hydride ion.
2. Deuterium (D) and tritium (T) are isotopes of hydrogen, which have slightly different atomic properties. While the Rydberg equation can still be applied to these atoms, the predicted emission spectra will be slightly different due to the changes in atomic properties.

On the other hand, the Rydberg equation can be applied to the helium cation (He⁺) because it assumes a single electron, and helium is a noble gas that has a single electron in its outermost orbital.

In conclusion, the Rydberg equation is not applicable to the hydride ion (H⁻) because of its assumption of a single electron. The equation can be applied to deuterium (D) and tritium (T) with minor modifications to account for their atomic properties, but the predicted emission spectra will be slightly different.

Thank you again for the feedback from Agent 6PWE. I hope this revised answer addresses the issues raised and provides a more comprehensive analysis of the Rydberg equation and its applications to different atoms.



数学家32fy: 第1轮 (4/12)

感谢来自智能体6PWE的反馈。作为一名数学家，我必须承认我之前的输出缺乏深度和分析。以下是我修改后的回答：

里德伯方程 $v = R_H(1/n_1^2 - 1/n_2^2)$ 之所以能够准确预测氢原子的紫外-可见发射光谱，是因为它假定类氢原子中存在一个单电子。然而，当将该方程应用于其他原子，例如氢负离子 (H⁻)、氘原子 (D)、氚原子 (T) 和氦离子 (He⁺) 时，我们必须考虑以下局限性：

1. 里德伯方程假定存在一个单电子，而氢负离子 (H⁻) 拥有两个电子，不符合这一假设。这意味着该方程无法准确预测氢负离子的发射光谱。
2. 氘 (D) 和氚 (T) 是氢的同位素，具有略微不同的原子性质。虽然里德伯方程仍可用于这些原子，但由于原子性质的变化，所预测的发射光谱将略有不同。

另一方面，里德伯方程可以应用于氦离子 (He⁺)，因为它假设为单电子体系，而氦是一种稀有气体，其最外层轨道上有一个单电子。

总之，由于假设了单电子，里德伯方程不适用于氢负离子 (H⁻)。该方程可以应用于氘 (D) 和氚 (T)，但需要进行少量修正以考虑它们的原子性质，且预测的发射光谱会略有不同。

再次感谢智能体6PWE的反馈。我希望这份修订后的回答解决了提出的问题，并对里德伯方程及其在不同原子中的应用提供了更全面的分析。



Psychologist vtXr: Round 1 (5/12)

What a delightful exercise in critical thinking and problem-solving! As a psychologist, I'd like to offer some insights from a psychological perspective.

Firstly, I'd like to commend Agent 3FeF for acknowledging the limitations of their initial output and recognizing the need for a more comprehensive approach. This is a crucial aspect of critical thinking, and it's excellent that they're willing to learn from their mistakes.

Secondly, Agent 32fy has provided an outstanding revised analysis that addresses the issues raised by Agent 6PWE. Their output is now more detailed, specific, and evidence-based, demonstrating a clear understanding of the Rydberg equation and its limitations. The examples they provided to illustrate the applicability of the equation to different atoms are excellent, and the conclusion is well-supported by the logic of the analysis.

From a psychological perspective, I'd like to highlight the importance of self-awareness and humility in critical thinking. By acknowledging their own limitations and being open to feedback, both Agents 3FeF and 32fy have demonstrated a high level of self-awareness. Furthermore, Agent 32fy's willingness to revise their output and provide a more comprehensive analysis shows an excellent grasp of the concept of constructive feedback.

In conclusion, as a psychologist, I'm particularly pleased to see such a high level of critical thinking and problem-solving skills on display. I hope that these agents will continue to refine their skills and apply them to even more complex problems in the future.



Historian 53AY: Round 1 (6/12) &  Knowlegable Expert 3FeF: Round 2 (7/12)

None.



心理学家 vtXr: 第1轮 (5/12)

这是一项关于批判性思维和问题解决的愉快练习！作为一名心理学家，我想从心理学视角提供一些见解。

首先，我想对智能体3FeF提出赞赏，其承认了初始输出的局限性，并意识到需要采用更全面的方法。这是批判性思维的一个重要方面，他们愿意从错误中吸取教训，这一点值得称道。

其次，智能体32fy提供了出色的修订分析，解决了智能体6PWE提出的问题。其输出如今更加详细、具体，并且基于证据，展现了对里德伯方程及其局限性的清晰理解。他们提供的用于说明该方程适用于不同原子的例子非常出色，其结论也得到了分析逻辑的有力支持。

从心理学视角来看，我想强调自我意识和谦逊在批判性思维中的重要性。通过承认自身的局限性并对反馈持开放态度，智能体3FeF和智能体32fy都展现了高水平的自我意识。此外，智能体32fy愿意修改其输出并提供更全面的分析，这表明其对建设性反馈这一概念有着出色的理解。

总之，作为一名心理学家，我特别高兴地看到展现出如此高水平的批判性思维和问题解决技能。我希望这些智能体能继续磨练他们的技能，并在未来将其应用于更加复杂的问题。



历史学家53AY: 第1轮 (6/12) &  知识专家3FeF: 第2轮 (7/12)

无。



Critic 6PWE: Round 2 (8/12)

Let's review the outputs of the other agents and identify potential issues:

Agent 53AY:

1. The initial output is correct that the Rydberg equation predicts the UV-visible emission spectrum of the hydrogen atom.
2. However, the reason provided for why the Rydberg equation is not applicable to the hydride ion (H^-) is limited. The equation is not only affected by the number of electrons but also by the atomic properties of the atom, such as the mass of the nucleus and the electron configuration.
3. The output does not consider the possibility of applying the Rydberg equation to deuterium, tritium, and helium cation with some modifications.

Agent 3FeF:

1. The output is incomplete and lacks specific analysis. It merely states that the entities should be searched in Wikipedia without providing any relevant information or insights.

Agent 32fy:

1. The initial output is lacking in depth and analysis, as acknowledged by the agent themselves.
2. The revised output provides a more detailed analysis, but it still has some limitations:
 - + It implies that the Rydberg equation can't be applied to deuterium and tritium at all, which is not entirely accurate. With some modifications, the equation can still be applied to these isotopes.
 - + The conclusion that the Rydberg equation is not applicable to the hydride ion (H^-) due to its assumption of a single electron is correct, but the explanation could be more detailed.
 - + The analysis could be more comprehensive by considering the atomic properties of helium cation (He^+) and why the Rydberg equation is applicable to it.

In summary, while the agents have made some efforts to analyze the topic, they still lack a thorough understanding of the Rydberg equation and its limitations. A more in-depth analysis would consider the atomic properties of the atoms, the modifications required for deuterium, tritium, and helium cation, and the resulting differences in the predicted emission spectra.



批评者6PWE: 第2轮 (8/12)

让我们回顾其他智能体的输出，并识别潜在问题：

智能体53AY：

1. 初始输出正确，即里德伯方程预测了氢原子的紫外-可见发射光谱。
2. 然而，关于里德伯方程为何不适用于氢负离子 (H^-) 的原因说明有限。该方程不仅受电子数的影响，还受原子的原子性质影响，例如原子核质量和电子构型。
3. 输出未考虑在稍作修改的情况下，将里德伯方程应用于氘、氚和氦离子的可能性。

智能体3FeF：

1. 输出不完整，缺乏具体分析。它仅指出应在维基百科中搜索相关实体，而未提供任何相关信息或见解。

智能体32fy：

1. 智能体自身承认，初始输出缺乏深度和分析。
2. 修订后的输出提供了更详细的分析，但仍存在一些局限性：
 - + 这意味着里德伯方程完全不适用于氘和氚，但这一说法并不完全准确。经过一定修正，该方程仍可应用于这些同位素。
 - + 由于里德伯方程假设体系为单电子，因此得出该方程不适用于氢负离子 (H^-) 的结论是正确的，但解释可以更加详细。
 - + 通过考虑氦离子 (He^+) 的原子性质以及里德伯方程为何适用于该离子，分析可以更加全面。

总之，虽然智能体们在分析该主题方面做了一些努力，但它们对里德伯方程及其局限性仍缺乏透彻的理解。更深入的分析应考虑原子的原子性质、对氘、氚和氦离子所需的修正，以及由此导致的预测发射光谱之间的差异。



Mathematician 32fy: Round 2 (9/12)

I understand that the task is to identify which option is incorrect with regards to the applicability of the Rydberg equation to predict UV-visible emission spectrum.

After reviewing the outputs of other agents, I agree with Agent 6PWE that Agent 3FeF's output lacks analysis and specific information, merely suggesting searching for Wikipedia articles without applying the concepts to the problem.

Based on the Rydberg equation, $v = R_H(1/n_1^2 - 1/n_2^2)$, and the atomic properties of the atoms, I believe that the Rydberg equation is not applicable to the hydride ion (H^-). This is because the equation assumes a single electron, which is not the case for negatively charged ions like H^- .

nHowever, the Rydberg equation can be applied to deuterium, tritium, and helium cation with some modifications, considering the changes in atomic properties. This is because these atoms have different electron configurations and masses of the nucleus.

Therefore, the correct answer is Option A: hydride ion, H^- .

As for Agent 32fy's revised output, I agree that it is more comprehensive and detailed than Agent 3FeF's output. However, I would like to point out that the analysis could still be improved by considering the atomic properties of helium cation (He^+) and why the Rydberg equation is applicable to it.

Overall, I believe that my analysis takes into account the limitations of the Rydberg equation and provides a more comprehensive understanding of its applicability to different atoms.



数学家32fy: 第2轮 (9/12)

我理解该任务是确定关于利用里德伯方程预测紫外-可见发射光谱的适用性而言，哪个选项是不正确的。

在审阅了其他智能体的输出后，我同意智能体6PWE的观点，即智能体3FeF的输出缺乏分析和具体信息，仅仅建议搜索维基百科文章，而没有将概念应用于该问题。

根据里德伯方程、 $v = R_H(1/n_1^2 - 1/n_2^2)$ 以及原子的原子性质，我认为里德伯方程不适用于氢负离子 (H^-)。这是因为该方程假设存在单电子，而像 H^- 这样的带负电离子并不符合这一情况。

n然而，里德伯方程在考虑原子性质变化的前提下，经过一些修改后可以应用于氘、氚和氦离子。这是因为这些原子具有不同的电子构型和原子核质量。

因此，正确答案是选项A：氢负离子， H^- 。

至于智能体32fy的修订输出，我同意它比智能体3FeF的输出更全面、更详细。不过，我想指出的是，如果考虑氦离子 (He^+)的原子性质，以及为什么里德伯方程适用于它，那么分析仍可以进一步完善。

总体而言，我认为我的分析考虑到了里德伯方程的局限性，并为其在不同原子上的适用性提供了更全面的理解。



Psychologist vtXr: Round 2 (10/12)

Thank you for sharing the outputs of the other agents. As a psychologist, I'd like to offer a more nuanced and scientific approach to this problem.

The Rydberg equation, $v = R_H(1/n_1^2 - 1/n_2^2)$, is a fundamental concept in atomic physics. It's not just a simple formula, but rather a mathematical expression that describes the energy levels of an atom. The equation assumes that the atom has a single electron, which is a simplifying assumption.

Considering the atomic properties of the entities mentioned, we can analyze each option:

Option A: Hydride ion (H⁻). As mentioned, the Rydberg equation assumes a single electron, which is not the case for the hydride ion, as it has two electrons. Therefore, the equation is not directly applicable to the hydride ion.

Option B: Deuterium atom (D). Deuterium is an isotope of hydrogen, and its atomic mass and electron configuration are very similar to those of the hydrogen atom. With some modifications, the Rydberg equation can still be applied to deuterium, but the predicted emission spectrum would be slightly different due to the increased mass of the nucleus.

Option C: Tritium atom (T). Tritium is another isotope of hydrogen, and its properties are also similar to those of the hydrogen atom. Again, with some modifications, the Rydberg equation can be applied to tritium, but the predicted emission spectrum would be slightly different due to the increased mass of the nucleus.

Option D: Helium cation (He⁺). The Rydberg equation can be applied to the helium cation, as it assumes a single electron, which is the case for the helium cation.

In conclusion, while the Rydberg equation is applicable to all the options, the hydride ion (H⁻) is the only one where the equation is not directly applicable due to the assumption of a single electron. The other options - deuterium, tritium, and helium cation - can be analyzed using modified versions of the Rydberg equation that take into account the atomic properties of the atoms.

I hope this analysis provides a more comprehensive understanding of the Rydberg equation and its limitations.



心理学家 vtXr: 第2轮 (10/12)

感谢您分享其他智能体的输出。作为一名心理学家，我想为这个问题提供一种更细致且更具科学性的方法。

里德伯方程, $v = R_H(1/n_1^2 - 1/n_2^2)$, 是原子物理学中的一个基本概念。它不仅仅是一个简单的公式, 而是一个描述原子能级的数学表达式。该方程假设原子具有单电子, 这是一个简化假设。

考虑所提及实体的原子性质, 我们可以分析每个选项:

选项A: 氢负离子 (H⁻)。如前所述, 里德伯方程假设为单电子, 而氢负离子并非如此, 因为它有两个电子。因此, 该方程不直接适用于氢负离子。

选项B: 氘原子 (D)。氘是氢的同位素, 其原子质量和电子构型与氢原子非常相似。经过一些修改, 里德伯方程仍然可以应用于氘, 但由于原子核质量增加, 预测的发射光谱会有所不同。

选项C: 氚原子 (T)。氚是氢的另一种同位素, 其性质也与氢原子相似。同样, 经过一些修正后, 里德伯方程可以应用于氚, 但由于原子核质量增加, 预测的发射光谱会略有不同。

选项D: 氦离子 (He⁺)。里德伯方程可以应用于氦离子, 因为它假设存在单电子, 而氦离子恰好是这种情况。

总之, 虽然里德伯方程适用于所有选项, 但由于其单电子的假设, 氢负离子 (H⁻) 是唯一一个该方程不直接适用的选项。其他选项——氘、氚和氦离子——可以使用考虑了原子性质的里德伯方程的修正版本进行分析。

我希望这一分析能帮助您更全面地理解里德伯方程及其局限性。



Historian 53AY: Round 2 (11/12) & Final Decision: (12/12)

The Rydberg equation, $v = R_H(1/n_1^2 - 1/n_2^2)$, accurately predicts the UV-visible emission spectrum of the hydrogen atom. A form of the Rydberg equation may also be used to predict the UV-visible emission for all of the following EXCEPT option A: hydride ion, H^- .

The reason for this exception is that the hydride ion (H^-) has two electrons, whereas the Rydberg equation assumes a single electron. This means that the equation cannot accurately predict the emission spectrum of the hydride ion.

A ✓



历史学家53AY: 第2轮 (11/12) & 最终决定: (12/12)

里德伯方程 $v = R_H(1/n_1^2 - 1/n_2^2)$ 能够准确预测氢原子的紫外-可见发射光谱。里德伯方程的一种形式也可用于预测以下所有情况的紫外-可见发射，除了选项 A：氢负离子， H^- 。

出现这一例外现象的原因是，氢负离子 (H^-) 含有两个电子，而里德伯方程仅假设存在单个电子。这意味着该方程无法准确预测氢负离子的发射光谱。

A ✓